UMEÅ UNIVERSITY

Parallel Kafka Producer Applications

Their performance and its limitations

Arvid Sundbom

Arvid Sundbom
Spring 2023
Degree Project: Master of Science in Computing Science, 30 credits
Supervisor: Jerry Eriksson
Extern Supervisor: Joel Sandman
Examiner: Andreas Theodorou
Master's Programme in Computing Science, 300 credits

Abstract

This paper examines multi-threaded Kafka producer applications, and how the performance of such applications is affected by how the number of producer instances relates to the number of executing threads. Specifically, the performance of such applications when using a single producer instance, shared among all threads, and when each thread is allotted a separate, private instance, is compared. This comparison is carried out for a number of different producer configurations and varying levels of computational work per message produced.

Overall, the data indicates that utilizing private producer instances results in higher performance, in terms of data throughput, than sharing a single instance among the executing threads. The magnitude of this difference is affected, to some extent, by the configuration profiles used to create the producer instances, as well as the computational workload of the application hosting the producers. Specifically, configuring producers for reliability seems to increase the difference, and so does increasing the rate at which messages are to be produced.

As a result of this, *Brod*, a wrapper library [56], based on an implementation of a client library for Apache Kafka [25], has been developed. The purpose of the library is to provide functionality which simplifies the development of multi-threaded Kafka producer applications.

Acknowledgements

Thanks to Jerry Eriksson for being a good supervisor, thanks to Joel Sandman and Magnus Norrby for their help with Nasdaq's internal systems, and thanks to Elias Häreskog for good games of table tennis, although he seldom won any.

Contents

1 Introduction **1**

1.1 Motivation 1

1.2 Research Question 3

1.3 Contributions of this Work 3

1.4 Evaluating Producer Performance 3

2 Background **5**

2.1 Distributed Systems 5

2.1.1 Challenges of Implementing Distributed Systems 5

2.1.2 Publish-Subscribe Systems 7

2.2 Parallel Software Systems 11

2.2.1 Parallelism and Concurrency 11

2.2.2 Why Parallelism? 12

2.2.3 Cores and Threads 13

2.2.4 Types of Parallel Systems 13

2.2.5 Issues Faced when Implementing Parallel Systems 14

2.2.6 Measuring Parallel Performance 16

2.2.7 Parallel Programs and Input/Output 17

2.3 Apache Kafka 18

2.3.1 Configuring Kafka for Performance 20

2.3.2 Acknowledgement Messages 20

2.3.3 Exactly-Once Semantics 21

2.3.4 Message Compression 21

2.3.5 Message Batching 22

2.4 OpenMP 23

3 Related Work **25**

4 Methodology **28**

4.1 Producer Evaluation Scenarios 28
4.1.1 Scenario One - Binary Data 28
4.1.2 Scenario Two - Trapezoids 28
4.1.3 Producer Allocation 30
4.2 Kafka Producer Configurations 31
4.3 Experimental Setup 32
4.4 Design of the Software Library 34

5 Results 37
5.1 Execution Times, Shared and Private 37
5.2 Message Latencies 41
5.3 Configuration-specific Results 45

6 Discussion 50

7 Conclusions and Future Work 54
7.1 Conclusions 54
7.2 Future Work 54

References 55

A Kafka Configuration Values 61

B Brod - Parallel Producer Support 62
B.1 Wrapper Function 62
B.2 Producer Pool 65
B.3 Performance 65

1 Introduction

This section introduces the general setting of the problem, as well as some motivation for why the problem at hand ought to be dealt with. Additionally, the research question this work aims to answer in order to lessen the problem at hand is also presented.

1.1 Motivation

The prevalence of miniaturized computers in today's society, made possible by the miniaturization and reduced cost of transistors as predicted by Moore [44] has enabled a wide range of technologies which previously only existed in works of science-fiction [3]. Among these technologies are such as the Internet, World-Wide Web, and Internet-of-Things. A common attribute of these technologies is that their usage generates vast amounts of data, with an estimated 60% of the global population having access and actively using the Internet in 2020 [5]. For example, in 2021 the social media platform Twitter alone generated petabytes of data each day, processing over 400 billion real-time events [14]. As such, the importance of harnessing data in order for businesses to stay competitive has increased significantly in the last decade [43, 42].

In addition to this, the continued development and spread of the Internet-of-Things, i.e. commonplace objects which are not primarily to be used as computers, but have small computer systems embedded within them, has increased from roughly 3.6 billion devices in 2015 to about 14.4 billion devices in 2022 [13]. In turn, this leads to additional data being generated in domains which have previously been inaccessible from a data viewpoint. Some examples of such IoT-systems are the *over-the-air* software updates and edge data management for autonomous vehicles, developed by Airbiquity [15], the series of smart fridges developed by Samsung [11], as well as personal mobile devices such as smart phones and smart watches, which also play a role in IoT systems.

One popular platform for developing applications which harness this data and make it available for use in software systems is Apache Kafka [31] [1]. Kafka is based on three constituent parts: *producers*, *brokers* and *consumers*, where producers send data to brokers, brokers host the data, and consumers read the data from the brokers. A central selling point of Kafka is the high performance that can be achieved by using it, both in terms of throughput and latency. This can be important for applications with strict constraints on time or resources, and can also be useful in order to enable these applications to scale in order to cope with the increased amount of

generated data.

It is common to deploy Kafka in a cloud environment, where computing resources and infrastructure are effectively rented from a cloud provider. As such, a rather common measure for increasing the performance of systems based on Kafka is to simply rent additional instances of computing resources in the cloud environment at hand, such as Elastic Compute Cloud (EC2) instances, in the case of Amazon Web Services (AWS) [53] [54].

Although this may have the intended effect of increasing performance, it may not be the most cost-effective approach for doing so. The reason for this is that these computing resources are commonly payed for on a monthly basis, so the addition of such instances each time performance needs to improve can quickly add up to high costs. Additionally, these computing resources often have multiple *Central Processing Unit (CPU)* cores. One may expect that in order to use the nodes in the most effective manner, and to keep the number of deployed nodes to a minimum, the application(s) running on each node would make use of all available cores. However, in the case of Kafka producers, which are the part of Kafka applications tasked with sending messages across a network, it seems that this is seldom the case.

While Kafka producers do not utilize multiple cores by default, they are thread safe, meaning it is at least possible to use them in applications executed using multiple cores [33]. Although executing producer applications in parallel is a possibility, there seems to be a lack of data regarding what impact this has on application performance, compared to sequentially executing applications. Furthermore, there seemingly does not exist any data nor consensus among the users of Kafka regarding the design of such applications. One of the primary issues one is faced with when developing a parallel producer application is the relationship between the number of threads of execution and the number of producer instances created. The official documentation for Kafka producers states that "...sharing a single producer instance across threads will generally be faster than having multiple instances". However, no data supporting this claim is provided. Furthermore, the community of Kafka users does not seem to agree on the accuracy of this statement, with some users claiming to gain some performance benefits by increasing the number of instances to more than one, and some supporting the claim made in the documentation [58]. Data supporting these claims is not provided in this case either.

Yet, indications of increased performance as a result of parallel execution exist for other parts of Kafka, namely consumers. For example, the Confluent parallel consumer library [2], which greatly simplifies the development of parallel consumer applications. From the data presented in the overview of the library, its use seemingly results in greatly improved application performance. However, no such library exists for Kafka producers.

1.2 Research Question

With all this in mind, it should be rather clear that a lot of uncertainty exists regarding parallel Kafka producer applications and their performance. Arguably the most central part of this uncertainty is the aforementioned relationship between the number of cores and the number of producer instances used to execute the application. As such, the research question of this thesis is

RQ: *Is the best performance, in terms of data throughput, for Kafka producer applications achieved by using a single producer instance, shared among the executing cores, or by using a separate, private producer instance for each core?*

In order to answer this question, the performance of parallel producer applications, using both a shared instance as well as private instances, will be evaluated in a number of performance tests, representing a variety of application characteristics.

An additional objective of this thesis is to develop a library which supports the development of parallel Kafka producer applications, as no such library exists yet. The use of either a shared producer instance or several private ones will be in line with the performance data to be presented in this work.

1.3 Contributions of this Work

The primary purpose of this work is to act as an investigatory study, the aim of which is to produce data and insight into the performance of Apache Kafka producer applications, utilizing Kafka producers, executed in parallel. While this data provides useful insight for all producer applications which may be executed in parallel, a specific aim of this investigation is to determine the validity of the claim made in the Kafka producer documentation, that using a shared producer instance among the threads of execution generally results in better performance than using multiple producer instances.

Additionally, a wrapper-library, based on an existing implementation of a Kafka client library, has also been developed. This library aims to support the development of parallel producer applications by providing functionality that reduces the amount of work required by developers aiming to create such applications. An overview of the library can be found at https://crates.io/crates/brod, which contains links to both the source code and documentation of the library. For additional details regarding the library and its implementation, see Appendix B.

1.4 Evaluating Producer Performance

In order to answer the presented research question by evaluating the performance of parallel producer applications, as well to guide the design and development of the library to be built, experiments which evaluate the performance of various producer

applications must first be carried out. However, before these experiments and their corresponding results are presented, some necessary background information related to *distributed* and *parallel* computer systems will be presented. After presenting this background information, previous work related to the work undertaken here will be presented, with an emphasis on the lack of data and insight into the performance of parallel producer applications.

With the background information presented, and the gap in the literature identified, the methodology used to fill this gap is then presented. The design of the experiments performed to gather data is presented along with some motivation for their design. In addition to this, the identified goals of the software library to be built are presented, along with the initial design decisions taken in order to achieve this goal. Finally, the resulting data of the performed experiments are presented and then discussed, with the resulting library presented in Appendix B.

2 Background

This Section presents the fundamental background information necessary to understand the contents of the following Sections of the thesis. Brief theoretical presentations of distributed systems and parallel systems are given, along with concrete instances of these, in the form of *Apache Kafka* and *Open MultiProcessing (OpenMP)*, respectively.

2.1 Distributed Systems

A *distributed system* is a computer system which is characterized by the constituent parts of the system being deployed on separate, networked, computers which coordinate only by passing messages to one another [23, Ch. 1]. The type of computers used in such systems varies from small, embedded devices used for purposes such as sensor networks, up to high-performance parallel supercomputers, used to carry out large, compute intensive tasks [4]. The reason for designing and constructing systems in a distributed manner is not that it is always easier or simpler to construct applications in this way, but because it is sometimes *necessary* to do so in order to construct a software system which meets the current and possibly future functionality demands of its users. The primary motivator behind building distributed systems is the concept of *resource sharing* [23, Ch.1]. Resources, in this context, can be both physical hardware resources such as printers and high-performing computational hardware as well as data in the form of files on a shared file-system, or web pages served to a client's web browser. This resource sharing also varies widely in scale, with a search engine used by millions of people on the web at one end, and a networked attached storage used by people in a single home at the other. No matter the scale of the distributed system, the part of the system which manages a given resource needs to define and expose a set of operations for utilizing the managed resource. By doing so, the resource in question is made available, in a controllable fashion, to the other parts of the system, enabling resource sharing.

2.1.1 Challenges of Implementing Distributed Systems

When designing and implementing distributed systems, there are a number of issues that arise, which are not as commonly encountered when constructing monolithic applications, due to the distributed nature of the system.

One notable issue is the increased complexity caused by each constituent part of

the distributed system being a system of its own, possibly a rather complex one. As such, each individual system brings with it additional complexity to the distributed system. Furthermore, the distributed system is made even more complex due to the fact that each system also needs to communicate with at least some of the other systems.

One source of aggravation which can be particularly difficult to fix in the distributed setting is that of *partial failures*. While parts of monolithic systems may also fail, by either crashing completely or simply behaving in a way other than that which is intended, this is effect is exacerbated in distributed systems [60, Ch. 1]. The cause of this is rooted in the fact that the system as a whole makes use of multiple smaller systems, which, in turn, make use of their own resources, such as computing hardware or networking infrastructure. As such, the number of points where a partial failure may occur is typically much larger in a distributed system than in a monolithic one, making the task of locating and fixing the cause of the error more difficult.

Another complication introduced by making use of a distributed system architecture is that of *scalability*. There are several dimensions along which a system can be said to be scalable [60, Ch. 1.2]. Here, we will focus particularly on the *size scalability* of distributed systems. Size scalability can be described as the capability of to improve the capacity of a system to handle an increased computational load without a noticeable degradation of performance. In large-scale distributed systems which are connected to the Internet, scalability becomes an important issue, because the service is, due to its Internet connection, available to a tremendous amount of potential users. As such, when and how users decide to utilize the system, and how many decide to do so at an overlapping time, can vary significantly. In order to handle this variability, and to be able to withstand the resource demands required usage is at its peak, distributed systems need to be scalable. There are several techniques which can be applied in order to resolve scalability issues.

The most direct way of scaling a system is what is referred to as *scaling up*, where the capacity of computational, storage, input-output, and network components of the system are increased by additional, or improved, hardware. Of course, the main drawback of this approach is the increased monetary cost of the improved hardware, whether it is purchased directly or rented, such as in a cloud computing environment. As such, it may not be possible to utilize this scaling strategy indefinitely.

Another approach to scaling is to *scale out*, which entails utilizing additional machines in the deployment of the system. The most basic approach to scaling out resembles the approach of scaling up, namely, deploying additional servers, and distributing requests among these. This approach is sometimes called *replication* [60, Ch. 1.2], as the service is replicated in multiple instances on separate servers. This approach may also make use of additional hardware resources, and it can as such also result in an increased monetary cost, though, typically a somewhat lower one.

2.1.2 Publish-Subscribe Systems

As is always the case when software systems of non-trivial scale are to be designed, the architecture chosen for the system, is of significant importance [35]. One key challenge of properly designing an overarching architecture for a software system is to manage the dependencies between the components of the system, decoupling the components of the system while making sure the system is still useful and fulfills its purpose [45, Ch. 2].

As software systems grow and therefore become increasingly complex to understand and manage, they are commonly partitioned into several components. As the system as a whole has been split into several parts, no such component provides the entire functionality of the system. Because of this, the functionality of the components need to be combined in order for the system to work as intended. This combination, where distinct components work together in order to achieve a higher degree of functionality, is the source of these dependencies. As such, it is generally not possible to design software systems with substantial functionality without introducing any dependencies. However, while dependencies are a necessary fact of software, they can also lead to prohibitive issues when developing software systems. In turn, can be caused by the fact that software systems tend to be actively used for prolonged periods of time. During this time, users' requirements of the system also tend to change. Therefore, the system itself also has to evolve, in order to match those requirements. This change may be hindered by inter-module dependencies that have been previously established and may be difficult to change or remove.

In distributed systems, it is possible to identify two dimensions of coupling between components which are especially important [60, Ch. 2.1]. These are *temporal coupling* and *referential coupling*. Temporal coupling, as the name suggests, involves a coupling in time between components. More concretely, if two components are temporally coupled, the system requires that the two components are up and running simultaneously in order to function properly. Referential coupling, on the other hand, requires that different components are explicitly made aware of each other, in a direct manner, when they need to communicate. As previously discussed, it is, in general, a goal of most software architectures to decouple the components of a system as much as possible. An architecture commonly used to reduce referential and temporal coupling is a *publish-subscribe architecture*, and systems which implement such an architecture are commonly called publish-subscribe systems or *distributed event-based systems* [23, Ch. 6.3].

Publish-subscribe systems are often implemented by the use of *middleware*. Middleware is typically described as a software layer between the operating systems and the applications developed to run on top of that operating system. Typically, middleware provides support for functionality which is commonly used in a particular application domain, simplifying the process of developing applications within that domain.

In the case of publish-subscribe systems, several types of middleware are available, such as RabbitMQ [10] and Apache Kafka [31]. A common aspect of publish- sub-

scribe middleware is that it provides extensive support for the development of the indirection between the parts of the distributed system which produce data, and the parts that consume data [60, Ch. 2.1] [23, Ch. 6.3]. Additionally, this middleware may also provide support for coordinating these parts, typically in the form of a separate executable, requiring no additional effort from the application developers. In this case, the producers of data send their data to this middleware executable, and the consumers of data notify the middleware which data they are interested in, typically referred to as *subscribing* to that data. A visualization of this situation is shown in Figure 1. In a distributed setting, these components are usually

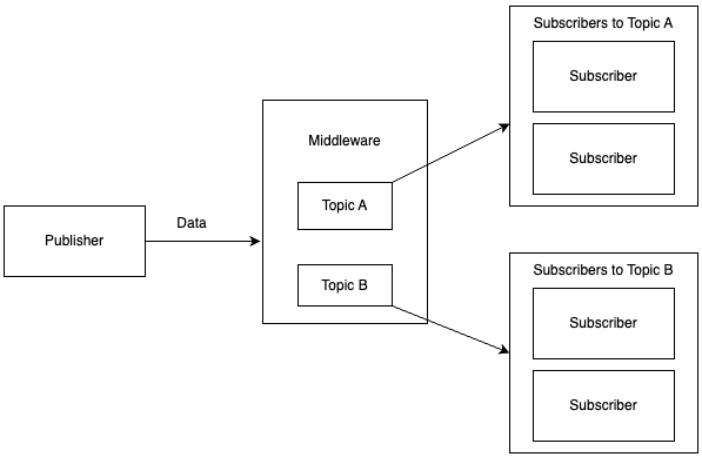


Figure 1: The typical components of a topic-based publish-subscribe system

deployed on separate computers, possibly with several computers dedicated to each part (producer, consumer, or middleware), as this removes any single point of failure, resulting in a more resilient system [23, p.249]. There are several approaches to how subscribers can define which data they are interested in; two of the most common are referred to as *topic-based* and *content-based* systems [60, Ch. 2.1] [23, Ch. 6.3]. In topic-based systems, subscribers declare which messages they wish to receive by specifying a particular *topic* in the middleware they wish to subscribe to. Producers can then publish data to the topics in the middleware when they have new data, the middleware takes care of making sure the data is delivered to the correct set of subscribers. These topics can be structured in an hierarchical manner in order to increase the expressiveness of the *subscription model* used by the system, by allowing consumers to use varying levels of specificity when declaring their topics of interest [23, Ch. 6.3]. On the other hand, in content-based systems, the subscribers specify their interest in data by composing a number constraints for each attribute they are interested in. Then, the middleware layer matches the attributes and values in the incoming data from the publishers with the constraints declared by each subscriber, and passes the data along to the matching subscribers. In general, topic-based subscription models offer greater expressive capabilities, allowing more intricate patterns to be declared and matched when mapping messages to consumers [23, Ch. 6.3]. However, this capability comes at the cost of increased complexity in the implementation of the system, as it requires the system to implement some mechanism for matching the published data with the correct subscribers [23, Ch.

6.3]. This mechanism usually takes the form of an algorithm implemented at the middleware layer, although several other approaches are possible, including that of *flooding*, in which **all** messages are forwarded to **all** subscribers, and each subscriber carries out its own filtering.

By delegating the responsibility of matching published messages with their corresponding subscribers, which are to receive the messages, the producing and consuming components of a system need not be tightly coupled. At least, they need not be referentially coupled. In order to decouple these components in a temporal manner, the middleware system also needs to offer storage capabilities [60, Ch. 2.2]. If the middleware is not capable of storing the messages received by producers, the consumers interested in receiving produced messages are necessarily required to be operating at the same time as the messages are sent to the middleware. However, if the messages sent by producers are stored, say on disk for example, at the computer(s) running the middleware, consumers are given at least some amount of time following the publication of the data to consume it. As such, the producers and consumers are, at least to some degree, temporally decoupled.

Performance of Publish-Subscribe Systems

It is possible to define the performance of publish-subscribe systems, or at least some of their components, with some mathematical rigour. In order to do so, we can analyze the behaviour of the system through the lens of a part of mathematics called *Queueing theory*. Here, we will be primarily concerned by the publisher part of publish-subscribe system, i.e. the parts of the system from which data are sent. Additionally, all values presented in this section are *long-term* averages, under the assumption that the system is in steady-state [37, Ch. 1]. Furthermore, another assumption made by this model is that the queue has an infinite capacity, so that additional requests can always be added to the queue. This simplifies matters somewhat, as this decouples the rate at which requests arrive from the current contents of the queue, allowing us to view the average arrival rate as a simple constant value.

Each publisher application in the system can be viewed as a queue and a corresponding process, as shown in Figure 2. The items in the queue correspond to requests asking the publisher to send data to the middleware part of the system. In turn, each such request is processed by the system. As such, additional items are only placed in the queue when requests are made while the publisher is currently processing a request, or if the queue already contains requests. After the publisher has processed a request, sending the requested message, it fetches another request from the queue, if one is available, and begins processing it.

This situation can be modelled mathematically. In order to do so, some notation must be introduced. The notation used here, along with the outline of the overall description will largely be based on the description of queueing systems given in [60, Ch. 1.2]. Most central to the mathematical description of this situation are the rate at which new message requests are made to the publisher, i.e. the rate at which they arrive at the queue, which we denote as λ, along with the rate at which the publisher is capable of processing such requests, denoted as μ. If the publisher is capable of

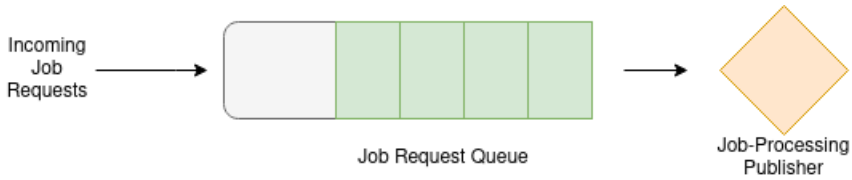


Figure 2: A visualization of modeling a publisher as a queueing system

processing μ requests per second (or any given time unit), we can also define the average time required for processing a requests as $\tau = \frac{1}{\mu}$ seconds. It should be noted that this is simply the time spent actively processing the request, and does **not** include the time spent in the queue. The value, L, for *latency* consists of the time spent in the queue as well as in active processing, is what we are working towards defining.

Building on the definitions established so far, we now model the performance of the system in a more thorough manner. Given a number of message requests, n, the fraction of time that there are n outstanding requests to the system, denoted p_n, can be calculated as

$$p_n = (1 - \frac{\lambda}{\mu})(\frac{\lambda}{\mu})^n$$

As an extension of this definition, the *utilization* of the system, defined as the fraction of time that the publisher has at least one request to process, can be calculated as

$$U = \sum_{n>0} p_n = 1 - p_0 = \frac{\lambda}{\mu}$$

Additionally, this definition also allows us to re-define p_n as

$$p_n = (1-U)U^n$$

With the utilization measure defined, the two key values which in turn determine the latency L can be defined. These are T, the throughput of the system, and $\overline{N}$, the average number of requests yet to be processed by the publisher. The definition of the publisher's throughput is fairly straightforward, and is based on the observation that when the system is busy, which it is for a fraction U of the time, it requests are processed at a rate of μ, and when it is idle, no requests are processed at all. As such, the throughput can be defined as

$$T = U \cdot \mu + (1-U) \cdot 0 = \frac{\lambda}{\mu} \cdot \mu + 0 = \lambda$$

i.e. the throughput equals the rate at which requests arrive at the publisher. The average number of unprocessed messages in the system, $\overline{N}$, equals the sum of the fraction of time the system contains n messages, for each value of n greater than or equal to 0. This observation can then be expanded to a definition based on utilization, using the second definition of p_n, as follows

$$\overline{N} = \sum_{n\geq 0} n \cdot p_n = \sum_{n\geq 0} n \cdot (1-U)U^n = (1-U)\sum_{n\geq 0} n \cdot U^n = \frac{(1-U)U}{(1-U)^2} = \frac{U}{1-U}$$

where the last two expansions make use of the fact that U can be assumed to be in the range $(0, \infty)$, as the publisher is assumed to be used at least once, with some overhead prohibiting it from always actively processing.

Finally, the two measures $\overline{N}$ and T can be used to calculate the average latency L for each request, where

$$L = \frac{\overline{N}}{T}$$

Then, using *Little's formula*: $\overline{N} = \lambda \cdot R$, which states that the average number of items in a system $\overline{N}$ is equal to the average arrival rate λ times the average time spent by the item in the system, R [37, Ch. 1]. This formula can be used to derive our final result:

$$L = \frac{\overline{N}}{T} = \frac{\tau}{1-U} \Rightarrow \frac{R}{\tau} = \frac{1}{1-U} \tag{2.1}$$

From this, two important facts can be observed. Firstly, in order to decrease the ratio between the time each request spends in the queue and the time it spends being processed, $\frac{R}{\tau}$, the utilization of the system needs to be relatively low. Crucially, the utilization must be kept from reaching values close to 1.0, as this results in a severe increase of the ratio's value, leading to a much larger average latency. Secondly, and perhaps most importantly, under the assumption that it is not possible to directly affect the rate at which new requests are sent to the publisher, λ, the only way to reduce the latency L is to increase the throughput T, which effectively means reducing the time required to process each request, τ. One potential approach to achieve this reduction, without actually increasing the rate at which requests are processed by a given publisher, is to instead make use of multiple publishers, processing incoming requests in *parallel*.

2.2 Parallel Software Systems

This Section gives a brief introduction and overview of parallel programs and the process of developing such programs.

2.2.1 Parallelism and Concurrency

In order to ease the discussion of parallel computer systems and the programming of such systems, the meaning of some core concepts within this subject should be briefly clarified. These concepts are a pair of similar but distinct ones: *parallelism* and *concurrency*.

Parallelism and concurrency are both concepts which relate to the execution of multiple computational tasks, but differ in significant ways, and should therefore not be used interchangeably [30, 48]. Concurrency refers to the act of composing multiple computations into separate threads of execution. This definition allows a situation in which only a single thread may be executing at any given time to still be an example of concurrency, as long as the computations to be performed are distributed among multiple threads. This can improve performance by allowing other

threads to execute while the currently executing thread is idle, such as when waiting for access to Input/Output (I/O) resources. For example, this approach is used by operating systems to give the *illusion* of executing multiple programs at once, even if only a single processor core is available and used for execution [51, Ch. 26] [47, Ch. 2.1].

Parallelism, on the other hand, necessarily involves the *simultaneous* execution of multiple computational tasks at the same moment in time [30, 48]. As such, in order to execute computational tasks in parallel, the executing system must have access to multiple *processing units*. Because of this simultaneous execution, parallelism is commonly used to increase the computational power available for solving some problem, either to solve it faster or to increase the size of problems which can be solved.

2.2.2 Why Parallelism?

Since the early days of digital computing, up to as recently as 2003, the performance of microprocessors increased by, on average, fifty percent per year [47, Ch. 1]. As such, software developers did not always have very significant incentives to work hard to improve the performance of their software. Instead, they could simply wait for the performance of the hardware running their software to improve. However, from 2003 onward, the rate of improvement of microprocessor performance has slowed dramatically. Between 2015 and 2017, the average yearly increase was just under four percent, far from the rate of improvement the computing world has previously been able to rely on [47, Ch.1].

A major reason for this decreased rate of improved performance is the "break down" of what is commonly referred to as *Dennard Scaling* [30, 26, 18]. Dennard Scaling says, roughly, that as the physical size of transistors get smaller, so does their power use, and, as a result of this, their operating frequency increases, resulting in increased performance of transistor-based integrated circuits. However, around 2006, this scaling "broke down", and the increase in transistor operating frequency slowed down, along with the performance benefits which resulted from it.

In spite of (or perhaps because of) this decreased rate of improvement, the need for increased computing capabilities remains great in large areas of modern society [47, Ch. 1] [52, Ch. 1]. For example, modern science and engineering work would likely not be possible without today's computing capabilities, as both areas rely on large quantities of numeric calculations, usually in the form of simulations, in order to carry out the required work within the field. In addition to this, it is likely that an increase in the available computational capacity would lead to further advances within these broad fields. In order to bring forth such an increase, the trend for the last twenty years has been to focus on *multi-core* processors [47, Ch. 1].

Such processors have two or more processing units, commonly referred to as *processor cores* or just *cores*, on a single integrated circuit. As each of these processing units can, at least theoretically, process their own stream of instructions, placing n processing units should, theoretically, lead to a speedup of a factor n. However,

this theoretical improvement factor is seldom met in practice. This is partly due to the fact that simply adding additional processing cores to the integrated circuit does not automatically lead to performance improvements. It is very rarely the case that applications developed for sequential execution on a single processing unit will see any performance improvements simply due to the presence of additional processing units. Unfortunately for software developers, changes, sometimes drastic ones, commonly have to be made in order to convert a sequential program into one which executes in parallel.

2.2.3 Cores and Threads

Another commonly occurring point of confusion when discussing parallel computer systems is the distinction between *threads of execution*, commonly referred to as just *threads* and *cores* [61]. While a core is a concrete, tangible hardware component in a computer, capable of executing instructions, the concept of a thread is somewhat more abstract. Roughly speaking, a thread represents a sequence of instructions to be executed, more or less sequentially. Technically, a program can be comprised of any number of threads, equal to or greater than one, regardless of the number of processor cores available on the system. In the context of this work, each core will at most be assigned a single thread of execution. As such, although it is not entirely accurate, in the rest this work, the terms core and thread will be used interchangeably.

2.2.4 Types of Parallel Systems

There are several ways to categorize parallel systems. One popular way in which parallel systems can be categorized is by what memory is accessible to each of the processor cores [47, Ch. 2.3].

Parallel systems are commonly categorized as either *shared-memory* or *distributed-memory* systems [47, Ch. 2.3]. In a shared-memory system, the cores all have access to the same set of shared memory, usually represented as a contiguous, addressable array. Typically, the cores and the associated memory are all physically present on the same physical board in shared-memory systems. As such, the cores have to share the physical hardware used to access the memory, the simplest of which is called a set of parallel wires called a *bus*. A consequence of this is that the performance of shared-memory systems can degrade as the number of cores increases, due to increased contention for use of the memory-access hardware between the cores. In shared-memory systems, any direct communication between cores is usually carried out by some core writing a value to a certain address, which is then read by the other cores.

By contrast, in distributed-memory systems, each core has its own, private set of memory which it alone is allowed to access. Typically, distributed-memory systems are composed of several distinct systems, called *nodes*, located on separate boards. Each of these systems may, in turn, be a shared-memory system in its own right. Communication between the nodes of a distributed-memory system usually takes the form of message-passing across a network. As such, the time required by the commu-

nication is typically significantly longer in distributed-memory systems, compared to shared-memory systems. However, since the nodes which constitute a distributed system do not have to share the physical hardware used to access the memory, as is the case for shared-memory systems, there is also no significant hardware contention to limit the number of nodes that can be used in a system. Therefore, the largest parallel systems are typically some form of distributed-memory systems.

2.2.5 Issues Faced when Implementing Parallel Systems

Unfortunately, it is quite common that devising a parallel solution to a problem is rather difficult, even if it is simple to design and implement a sequential solution to the problem at hand [47, Ch. 1]. One reason for this is that a goal of implementing parallel programs is to utilize, to the fullest possible extent, the available hardware *in an effective manner*. What constitutes an effective use of hardware may be difficult to define precisely. However, a reasonable basis for an effective use is that the presence of the additional hardware contributes to an increase in some quality of the produced result, or contributes to solving the problem faster than was possible with a serial solution [30]. For example, additional hardware may shorten the time required to carry out some task, such as compiling code in a software project, or increase the quality of the produced result, such as the number of pixels rendered to a screen.

In turn, underlying the problem of effective hardware resource usage is the problem of load-balancing, dividing the work to be carried out among the available hardware resources, such that they are used in an effective manner. In general, there are two approaches to dividing computational work: *data parallelism* and *task parallelism* [47, Ch. 1].

Data parallelism entails the division of computational work by the division of the data that work is to be carried out on. If the computation to be carried out on a given part of the data does not depend on the results of the same computation on a different part of the data, then these computations can be carried out in parallel. As such, if this is indeed the case for the set of data at hand, a parallel solution can be implemented by dividing the data among the available cores. If the cores are equal in terms of performance, and the work to be carried out equally complex for each core, the data should ideally be divided equally among the cores, so that each one is assigned the same amount of data.

Task parallelism, on the other hand, can be described as the situation which arises when there are several independent tasks, each processing their own set of data, simultaneously. Within each task, there may be dependencies between the data items worked upon. As such, it may not be possible to simply divide the data to be processed among the available cores as in data parallelism. Perhaps the most obvious example of task parallelism are two completely separate programs running on a single computer, which has two processor cores available. In such a situation, each program could be assigned to a separate core, allowing each program to fully utilize

the processing power of the core it has been assigned to. Of course, task parallelism is also often applicable within single programs. For example, one task within a program might be to perform some set of complex calculations, while another is to update the user interface to show the results of these calculations. In this case, this single program could be executed using two cores, one performing the calculations and one updating the user interface, potentially increasing the performance of the program by reducing contention for hardware resources among the two tasks.

Regardless of which kind of parallelism is utilized in a program, it is usually necessary for the cores executing a parallel program to coordinate their actions with each other. This coordination can be described by the three categories *communication*, *load-balancing*, and *synchronization* [47, Ch. 1.4].

In the context of core coordination, communication often refers to the communication required to produce a final solution to the problem, from the partial solutions produced by each core. This situation usually occurs when such partial solutions need to be either combined or compared with each other in order to produce the solution to the original, whole problem. A simple example of such a situation is when a sum or any result produced by an associative, binary operator, is to be calculated, with each core calculating a partial sum. Then, in order to produce the total sum, every partial sum needs to be combined, which requires the cores to communicate with each other, in some more or less clever manner.

Load-balancing, as explained previously, is the act of dividing the computational work among the available hardware resources in a balanced manner. There are a number of possible approaches to load balancing. However, in shared-memory systems, a common approach is to base the division of work on the *rank* of each core. The rank of a core is simply a unique integer assigned to each core, used to identify it among the available cores. If `n` cores are available on the system at hand, each of these are assigned a unique integer in the range $(0, n-1)$. When executing the program, each core can then identify their rank, perhaps by calling a function which returns the rank of the calling core, and then, in the case of data parallelism, calculate the partition of the data which they are to process. One measure which can be used in order to determine how evenly the load is distributed between threads is the ratio between the maximum and minimum CPU times of all the threads [59]. If this measure is equal to 1.0, the load is perfectly balanced between the available cores. The larger the measure is, the more unbalanced the load.

Finally, the purpose of synchronization is to, at least approximately, make sure that the cores are executing the same instruction, in the case of data parallelism. Normally, each core executes the instructions they are given independently of the instructions executed by the other cores. However, there are situations in which it is beneficial, or perhaps even necessary, to control when cores are allowed to execute certain instructions. One of the most common situations where this is necessary is when inter-core communication is also necessary, specifically to combine the partial solution produced by each core into a final result. In such situations, synchronization is commonly used to make each core halt execution after producing its partial result, in order to wait for any other core which might not have produced its partial

result yet. This is done because it might not be possible to combine the partial solutions to a complete solution before *all* partial solutions are available.

2.2.6 Measuring Parallel Performance

As previously mentioned, the prime underlying reason for writing parallel programs is to achieve greater performance than what is possible when executing programs on a single core. As such, metrics for measuring any such performance improvements would be of great use in order to determine how well a parallel program performs. As it happens, there are two fundamental metrics commonly used for this precise purpose. These two metrics are *speedup* and *efficiency*, both of which are related to the execution time of serial and parallel programs.

Speedup is commonly referred to as the quotient between the execution time of the serial implementation, denoted T_{serial}, and the execution time of the parallel implementation, $T_{parallel}$ as

$$S = \frac{T_{serial}}{T_{parallel}}$$

where S denotes speedup. Simply put, the speedup the factor by how much faster the parallel program executes, compared to the serial implementation. If the parallel program is executed on n cores, and the achieved speedup is n times faster, the parallel implementation is said to have *linear* speedup. However, this is seldom achieved in practice, as the parallel implementation usually introduces some form of overhead compared to the sequential implementation, typically in the form of coordination between the processor cores.

The other metric of parallel program performance is efficiency, which is based on the speedup metric. Usually, efficiency is measured according to the formula

$$E = \frac{T_{serial}}{(p \cdot T_{parallel})}$$

where p denotes the number of cores used to execute the program, and E the measured efficiency. A somewhat more intuitive description of efficiency is that it measures the fraction of the total parallel execution time that each core spends actively performing computational work. It can therefore be said that efficiency is a measure of to which degree the additional computational power introduced by each core is used to solve the problem at hand. As such, it is generally the case that the higher efficiency a parallel implementation is able to achieve, the better.

Unfortunately, the efficiency of a parallel implementation tends to decrease as the number of cores used to execute it is increased. One reason for this is that parallel programs usually contain some sequences of instructions which must be performed sequentially in order to produce correct results. Such instruction sequences are commonly referred to as *critical sections*. If such critical sections constitute a fraction **r** of the total program, it is impossible, no matter how many cores one has available, to achieve a speedup greater than $S = \frac{1}{r}$, a limitation commonly referred to as *Amdahl's Law* [17]. As efficiency is strongly related to speedup, this also limits the achievable efficiency. However, one needs to take into account that the size of

a program's critical sections are typically static, they do not depend on the size of the current problem instance. As such, the fraction occupied by critical sections typically decreases as the size of the problem to be solved is increased. As a result of this, the values of speedup and efficiency which are possible to achieve typically increase as larger problem instances are solved. Furthermore, as large problem instances typically take the longest to solve, they typically benefit the most from any achieved performance improvements.

2.2.7 Parallel Programs and Input/Output

One complicating factor when developing parallel programs, perhaps especially when doing so on shared-memory systems, is how I/O resources are used by the processing cores of the system [47, Ch. 2.5, 7.1]. Sequential programs, executed on a single core, only need to contend for these resources with other programs, when multiple programs are executing simultaneously on the same computer system. Typically, this contention is resolved by the operating system, if one is available on the system used. As such, it is rarely necessary for developers of sequential programs to care in great detail about how the contention for I/O resources is resolved.

On the other hand, the situation becomes rather more complicated when multiple cores are executing in parallel, so much so in fact, that entire books have been written about the subject [49]. This is because in addition to contending with other programs executing on the same system, the processor cores themselves *may* need to contend with each other for these I/O resources. This may be the case, but does not necessarily have to be, depending on how access to I/O resources are distributed among the cores. Most tools for parallel programming allow the programmer to explicitly control which cores are granted access to I/O resources.

A common practice is to only grant one core, the "parent" core which initiates execution on the other, "child" cores, access to certain I/O resources, while others are shared by all active cores. For example, on Unix systems, it is fairly common to only grant access to the *stdin* input stream to the parent core, for reading input from the user, while allowing all cores to output data to the *stdout* output stream. This means that the cores must contend for access to stdout and, as a result of this, that the order in which the cores write data to the output stream is unpredictable.

This is also the case for other I/O resources, such as access to files and network interfaces. Access to these resources typically has a larger impact on application performance than that of stdout and stdin, for example by determining how fast data can be written after being processed, either to file or over a network, before processing the next set of data. This impact is further increased in the context of parallel programs, as contention for the resources increases. In fact, if this contention is large enough, the limiting factor of what program performance is achievable may be determined by the performance of the I/O resources, not the performance of the actual processor cores. Such a situation is likely to result in poor measures of speedup and efficiency. Furthermore, increasing the number of cores is unlikely to resolve the situation and improve application performance, as it does not address the bottleneck rooted in limited I/O performance.

2.3 Apache Kafka

Apache Kafka is a distributed event-streaming platform, designed to handle extremely high volumes of event data, based on a publish-subscribe architecture [55, Ch. 1] [50]. Kafka operates at the middleware layer in distributed systems, as discussed in Section 2.1.2. Described at a high level, Kafka consists of three separate parts which are connected by passing messages between each other. These can be seen in the diagram shown in Figure 3. The three parts are *producers*, which constitute the publish-part of the publish-subscribe architecture, *consumers*, which subscribe to the data generated by the producers, and *brokers*, which are positioned in-between the producers and consumers [55, Ch. 1].

Brokers act as servers in Kafka, receiving messages from producers, stores these messages on disks, and handles meta-data related to the consumption of the messages, such as assigning *offsets* to the received messages. In addition to this, brokers also interact with the consumers of data, by handling fetch-requests sent by the consumers, responding with the appropriate data generated by producers. Typically, brokers are configured to act in coordination together with other brokers, as a *cluster* of brokers [55, Ch. 1]. Within each such cluster, one of the constituent brokers is chosen to be the leader of the cluster, and is tasked with managing the cluster in an administrative manner, such as by keeping track of whether any of the other brokers fail, and distributing the data that is written and read in an even manner among the brokers. An illustration of this is shown in Figure 4.

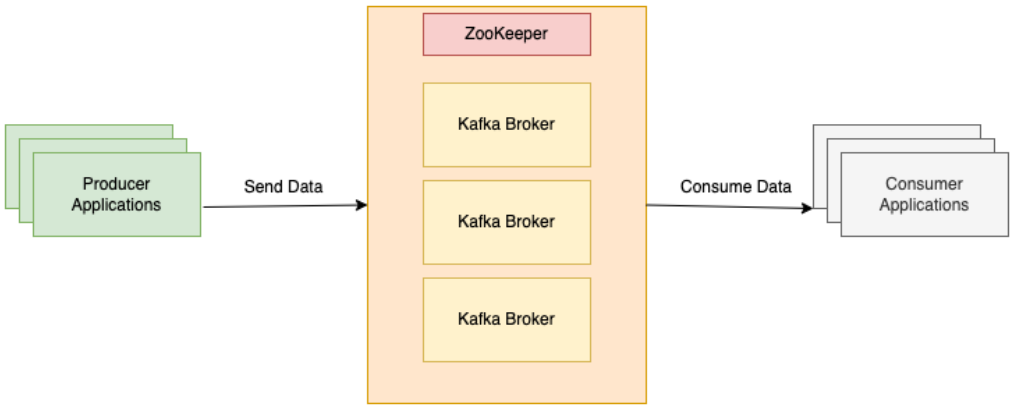


Figure 3: A high-level view of Kafka components

Although brokers are *positioned* at the center of Kafka, in-between producers and consumers, they are not, typically, the center of attention for developers of applications which use Kafka [32]. Instead, the main focus when using Kafka typically falls on the concepts of *topics* and *partitions*. Within Kafka, messages are categorized by topics; when producers write messages, they do so to specific topics. In the same way, consumers subscribe to particular topics in order to read the data published to each topic by producers. These topics are *partitioned* among the brokers within Kafka clusters [32]. What this means is that the data written to a given topic by

one or more producers is divided among the brokers that make up the cluster. For each partition of a given topic, a broker within the cluster is chosen as the *leader* of that partition. Copies of each partition are kept among the other, *follower* brokers. These followers replicate the state of the leader partition, providing the system with redundancy in case the leader broker fails. Typically, the producers write messages directly only to the leader of the partition, and these messages are then replicated to the followers of the leader for that specific partition. However, consumers may fetch data from any broker which has a copy of the partition in question, regardless if it is the leader or a follower. Any number of producers can simultaneously produce data to a particular topic, and the same goes for consumers reading data from a given topic. Furthermore, a single producer can write to more than one topic at a time, and a single consumer can read from multiple topics, as well.

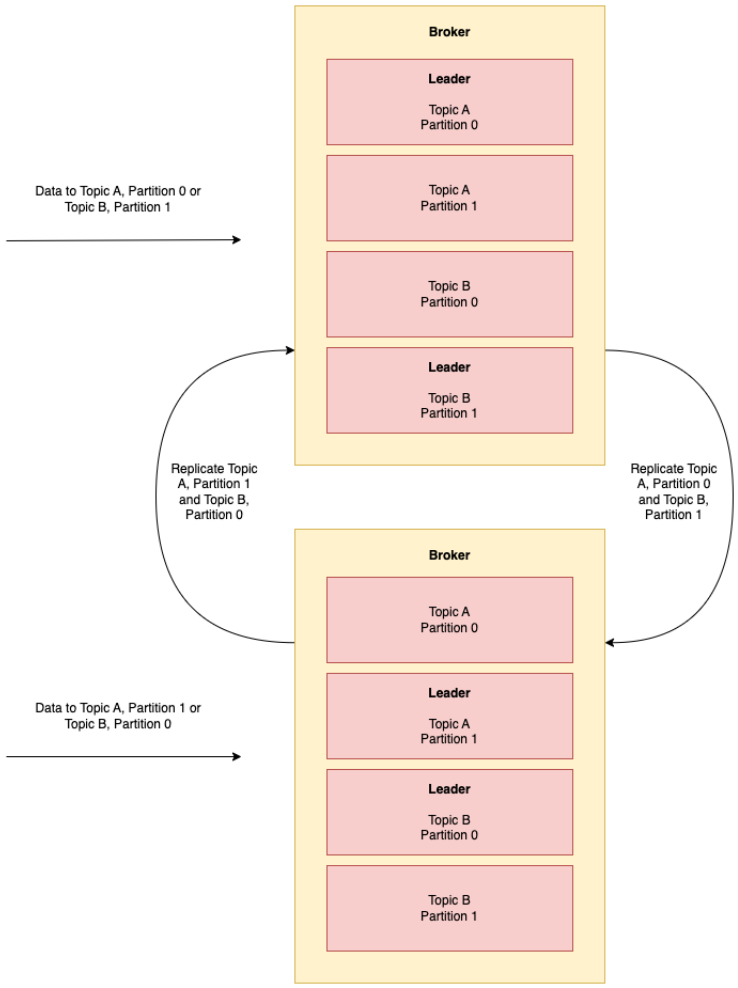


Figure 4: The internal division of topics and partitions within and between Kafka Brokers

Put simply, topics can be compared to folders on the file system of a computer, with messages, representing events which the system cares about, being similar to files within a specific folder [32]. However, in contrast with files and folders on a file

system, topics within Kafka are, in principle, *append-only*, meaning it is exceedingly difficult to remove data to a topic once it has been written to one by a producer [55, Ch. 1]. However, as computer storage is never infinite, keeping data around forever would eventually lead to the depletion of the system's storage. As such, a *retention period* can be configured, which specifies when messages sent to a broker may be deleted [55, Ch. 2]. This retention period can be specified in either the minimum period of time each message needs to be retained, the total size of message-data retained, specified in bytes per partition, or both. In the case where both these limits are configured, messages will be deleted when *either* of the set limits are reached. As such, data may be deleted before the set time limit if the size limit of the data is reached, and vice versa if the time limit passes before the data reaches the set size limit.

2.3.1 Configuring Kafka for Performance

Although at a high level, Kafka can be seen as having only a handful of constituent components, the configuration options available to these components are very numerous. For example, at the time of writing, there are over eighty different key-value pairs which can be assigned values in the configuration-map for consumers and producers, respectively, when using the standard Java library [7, 8], and over one hundred when using the C library [9]. These configuration options give developers control over aspects such as the maximum allowed size of messages, if brokers should respond to messages sent by producers, and what algorithm to use for reassigning topic partitions to remaining brokers if one fails. Furthermore, the number of brokers used in a cluster and how the partitions of a topic are distributed among these brokers are also important configuration factors when using Kafka. Several of these options have significant effects on the performance and reliability of the system as a whole. Therefore, when evaluating systems which utilize Kafka, it is important to take these configuration options into consideration, along with the intended use-case and requirements, on both performance and reliability, of the system. The configuration options with the most significant impact on producer performance, in terms of throughput, are presented in the following four Sections.

2.3.2 Acknowledgement Messages

The required number of acknowledgement messages, commonly denoted as *acks*, is defined by the setting suitably named `acks`. This setting has three possible values: 0, 1, and *all*, with increasing number of *acks* required for a message to be considered as successfully sent. With *acks* set to 0, the producer does not wait for *any* reply from the broker before assuming that the message was sent successfully. If something happens which causes the message to be dropped before reaching the broker, the producer will not be informed of this. Therefore, this setting is the least reliable of the three. The upside of setting *acks* to 0 is that the producer does not need to wait for a response from the broker before continuing to send the next message,

possibly improving performance. If *acks* is set to 1, only the leader replica of the partition the message is sent to needs to respond for the message to be considered successfully sent. This offers a higher degree of reliability than setting *acks* to 0, but is still not completely reliable. If something happens to the partition leader, causing it to crash before the message in question can be replicated, the message will still be lost. Finally, setting *acks* to *all* will require, as the name suggests, all replicas to respond with an acknowledgement before a message can be considered as successfully sent and received. This improves the reliability compared to both settings 0 and 1, but introduces even more of a performance overhead, in the form of latency, than either of the other settings.

2.3.3 Exactly-Once Semantics

One important feature offered by Kafka is that of *Exactly Once Semantics (EOS)* [55, Ch. 3, 8]. As the name suggests, with EOS enabled, Kafka guarantees that if a message is delivered from a producer to a broker, it will be accepted by the broker only once. EOS is made possible by two key underlying features: idempotent producers, and transactional semantics [55, Ch. 8]. As we are primarily concerned with producer performance, only the former will be explained.

The guarantee offered by idempotent producers is that any messages which are sent multiple times by a producer due to its built-in retry mechanisms will be processed by the receiving broker only once [55, Ch. 8]. This is done by the addition of a unique ID to each producer instance as well as to each message sent by such instances. When receiving messages, brokers inspect these ID's and compare them with those of the most recently received messages, which are stored in-memory by the brokers. If a message is received with the same producer and message ID as those of a stored message, the newly received message is simply discarded. As such, the brokers can assure that messages are received at most once.

In order to enable the other "half" of the *exactly* once guarantee, namely *at least* once, a requirement for enabling idempotent producers is that the required acknowledgement messages for the producer instance is set to `all`. One other, less strict, limitation is also placed on another configuration setting, which limits the number of allowable outgoing requests which have not yet received a response. A further limitation of idempotent producers is that they do not recognize the content sent with each message. If the same message is re-sent "manually" by the application and not by the mechanisms built into Kafka, the idempotency guarantee does not hold.

2.3.4 Message Compression

There are four types of compression available in Kafka. These are `gzip`, `snappy`, `lz4` and `zstd`. The main difference between these in terms of performance characteristics is what level of compression they offer. Higher levels of compression result in smaller message sizes, lessening the amount of bandwidth required to send each message. However, the actual compression of the message requires additional computational work performed by the processor. As such, which compression type to use becomes a

trade-off between CPU usage and compression time, and bandwidth usage. In terms of this trade-off, the compression types `snappy` and `zstd` are very similar [27]. An overview of the characteristics of each of these compression types is given in table 1.

Table 1 Characteristics of three compression algorithms available in Kafka

Compression Type	CPU Usage	Compression Speed	Network Usage
`Gzip`	High	Slow	Low
`Snappy`	Medium	Medium	Medium
`Lz4`	Low	Fast	High

2.3.5 Message Batching

When using the C Kafka library, *librdkafka* [29], there are four settings which primarily affect how messages are combined into batches when sent by producers, which in turn has a large effect on the throughput achieved by said producers. These settings are `linger.ms`, `queue.buffering.max.kbytes`, `batch.size` and `batch.num.messages`, and they are strongly related to one another.

Arguably the most important of these settings is `linger.ms`, as this setting determines the maximum amount of time, in milliseconds, a producer will wait for additional messages to accumulate before transmitting the current set of messages to the target broker(s). If this configuration option is assigned a value of zero, no message batching will be carried out whatsoever, and the configured producer will transmit messages as soon as they are placed in the producer's message queue. However, if this option is assigned a value greater than zero, and less than the maximum value of 900 000, i.e. nine seconds, then the producer will wait *at most* that long before transmitting the batched messages. However, messages may be transmitted before this set time limit, if the configured value of the option `queue.buffering.max.kbytes` is reached. The default value of `linger.ms` is five milliseconds.

The maximum size of total sum of the sizes of the messages in the producer's message queue is controlled by the configuration option `queue.buffering.max.kbytes`, which assigns this maximum size in kilobytes. If this limit is reached, the producer will begin the message transmission process, regardless if the `linger.ms` limit has been reached. The value assigned to this option by default is 1 048 576 kilobytes, i.e. 2^{20} kilobytes, and the maximum allowable value is 2 147 483 647, i.e. 2^{32}, kilobytes.

Two configuration options that are similar to `queue.buffering.max.kbytes` are `batch.size` and `batch.num.messages`. These control the maximum allowable total size of each batch of messages sent by a producer. The value assigned to `batch.size` determines the maximum *size*, in bytes, of all messages added into a single batch of messages, while the value assigned to `batch.num.messages` limits the *number* of maximum messages allowed in a single batch. These options both have equal priority and, as a result of this, whichever value is reached first will limit the size of the message batch. The default number of messages allowed in a batch is 10 000, with a maximum configurable value of 1 000 000. For the size of the message batch in bytes, the default value is 1 000 000 bytes, with the maximum being 2 147 483 647,

i.e. 2^{32}, bytes.

The reason these four settings have such a large impact on the throughput achieved by producers is because of the performance impacts of *message batching*. As is rather common when discussing computer systems, processing items in batches results in a higher throughput. In the case of Kafka, this is primarily because I/O overhead incurred by sending messages over a network, such as the round-trip-time required for messages to reach brokers and their response to reach the producer, is amortized compared with that which would occur if messages were sent one by one [22].

2.4 OpenMP

Open MultiProcessing (OpenMP) is an *Application Programming Interface (API)* specification for programming parallel applications on shared-memory systems [47, Ch. 5] [52, Ch. 7]. One major reason behind the development of the OpenMP standard was to make the development of parallel programs easier for software developers [47, Ch. 5]. Additionally, one goal of the development of the OpenMP standard was to enable developers to incrementally parallelize existing sequential implementations of programs, without requiring significant parts of the implementation to be re-written. As such, the level of abstraction at which programs are developed using OpenMP is higher than that of other parallel-development tools, such as pthreads. Because of this, the additional amount of code necessary to implement a parallel implementation, compared to a sequential one, is often significantly smaller for implementations based on OpenMP compared with implementations based on other tools.

With other parallel implementation tools, such as pthreads, it is often necessary for the programmer to explicitly specify what computational work should be carried out by each thread, along with adding additional code for initializing and coordinating the threads [47, Ch. 4.2, 5]. However, OpenMP makes use of *pragmas* in C and C++ implementations, and corresponding *directives* in implementations using Fortran. The remainder of this text will assume the implementation uses C or C++, and therefore also pragmas.

In C and C++, pragmas are directives to the compiler which allows the programmer to convey information beyond that which is possible using only the language itself [19, App. A]. In OpenMP, pragmas are used to specify to the compiler which blocks of code should be executed in parallel, along with additional information regarding the details of that execution [47, Ch. 5.1]. The most basic of these pragmas is the directive `#pragma omp parallel`, followed by a block, enclosed by curly brackets, containing the code to be executed. This directive informs the compiler that the code contained within the following block should be executed in parallel. By default, OpenMP will determine the number of threads used to execute the block on its own, typically as many threads as there are available cores. However, the number of threads to use can also be explicitly specified. One way to do this is to extend the OpenMP-pragma at hand with the clause `num_threads(N)` where `N` denotes the number of threads which should be used.

One central concept in OpenMP is that of *shared* and *private* variables [47, Ch. 5.3]. In this context, a shared variable is one which is accessible by all executing threads, while private variables are thread-local, with a separate value for each thread. By default, any variable declared outside a block annotated with `#pragma omp parallel` will be a shared variable, and any variables declared within such blocks are private. Additionally, the thread-visibility of variables can be further controlled by the `shared` and `private` clauses. These clauses are written in conjunction with the `parallel` pragma, and, when given a list of variables as an argument, ensures that the supplied variables are given the specified visibility among the threads. For example, if a variable which is declared before a block to be executed in parallel is specified to be private in the `parallel` pragma, each thread will be given its own local copy of the variable when execution of the parallel block starts. The exception to this is the parent thread, which was executing before the parallel block was encountered, and therefore simply retains its already existing copy of the variable.

Within OpenMP, the scope of variables is the main tool provided to the programmer for avoiding situations like race conditions, which occur when multiple threads attempt to access the same, shared resource, at least one thread attempts to update the value associated with this resource, and the (possibly erroneous) outcome of these operations depends on the timing of the threads [52, Ch. 7.1], [47, Ch. 2.4]. Furthermore, this mechanism also enables programmers to avoid performance-degrading situations such as *false sharing* from occurring [47, Ch. 2.10]. It is therefore of general importance, for the sake of both correctness and performance, that the visibility assigned to variables is well-thought out.

3 Related Work

The development of parallel programs has become increasingly popular since the "power wall" was reached, limiting the performance attainable with a single processor core [24][p.40-46] [20]. In line with this, attempts at improving the performance of Kafka-based applications using parallelism have been made. However, these attempts have been primarily focused on aspects other than that of parallel producers. For example, one of the most prevalent efforts of improving the performance of Kafka using parallelism is the development of the *Confluent parallel consumer* [2]. The Confluent parallel consumer is a software library with an *Application Programming Interface (API)* which enables software developers to create Kafka consumer applications which consume messages in parallel, using a single consumer. This example is, arguably, conceptually fairly similar to the development of a library and API to aid the development of parallel producer applications. Nevertheless, at the time of writing, no such library exists, which is the topic of this work.

Beyond this, the primary focus on parallelism related to Apache Kafka is aimed at parallelism by scaling the number of brokers used in clusters, partitioning topics among them [50]. While this mechanism may be used in order to increase the overall performance of Kafka clusters, it relies on the ability to increase the computational power of the cluster by adding additional computational capacity, in the form of broker nodes, to it. As such, this approach can be said to be conceptually "orthogonal" to that of utilizing multiple threads of execution to execute a single producer or consumer application, which instead can be seen as an attempt to utilize the already available hardware to the fullest possible extent. Furthermore, as these two approaches are rather different, they could seemingly be used in conjunction with each other.

A performance benchmark of the scalability of a number of stream processing frameworks is presented in [38]. However, this evaluation only focuses on scaling the systems running the frameworks by adding additional computing nodes to cloud-computing based systems. As such, this is yet another example of increased performance by parallelism due to additional hardware, and is considerably different from multi-threaded Kafka applications.

A comparison between Kafka and *RabbitMQ* [10], another distributed publish/subscribe-based platform, is carried out in [28]. In addition to comparing the features unique to each platform, the performance of each platform is also compared, measured in both latency and throughput. In terms of latency, both platforms can deliver low-latency performance, at least in situations where delivery guarantees are not maintained. However, if at-least once delivery is required, the latency of Kafka increases significantly, almost doubling, while the latency of RabbitMQ remains roughly unchanged.

When comparing throughput of the systems, RabbitMQ achieves a higher throughput in the most basic setting, where only one broker is used, maintaining one partition for one topic; with the corresponding setting for RabbitMQ (one exchange and one message queue). In both cases, one producer is used to send data, and another consumer is reading it. While RabbitMQ does perform better in this simple setting, this does not apply for larger, more complicated systems.

As the systems under test are scaled up, the system based on Kafka is able to achieve a significantly higher throughput than the system using RabbitMQ, although the performance of RabbitMQ also increases somewhat as the system is scaled up. As such, for situations in which the decisive performance aspect is throughput rather than latency, and it is possible to scale the system at least somewhat, Kafka seems to be the better choice.

A performance benchmark of Apache Kafka is presented in [21]. In this benchmark, both the throughput capability as well as the achieved message latency are measured. However, these measurements are only performed with a single, fixed producer configuration, and does not measure how the performance varies along with the producer configuration. Furthermore, and perhaps more importantly, the benchmark does not attempt to utilize parallelism on the system running the producers in order to increase their performance.

A performance test which does examine how producer performance, in terms of throughput, is affected by changes in the configuration of the producer is presented in [16]. In this test, the values of four configuration parameters which have a large effect on producer performance are varied, and the resulting change in producer throughput is measured. However, this benchmark has some limitations. For example, only one of the compression types available in Kafka is tested. Furthermore, the range of values for the other three parameters under examination is rather limited. In addition to this, there is no attempt to utilize multiple threads of execution in order to produce messages in parallel.

Some unofficial, rather basic tutorials for how Kafka producers can be used in parallel, on a single system, exist [46] [12]. However, these typically only contain brief code examples showing how producers can be used among several threads and provide almost no information regarding how the producer configuration or computational task may affect how parallelism may best be applied, or whether it is beneficial whatsoever. One common piece of information regarding parallel producers, which appears in the official documentation for Kafka producers [33], is that sharing a single producer among several threads of execution will generally result in better performance than creating several producers, such as creating one producer per thread, for example. However, typically, no performance data is presented to support this claim.

Overall, there exists limited development support and information surrounding parallel applications that are using Apache Kafka producers. Kafka does not provide native support for executing producers in parallel in the form of programming constructs supplied by the platform. However, Kafka producers are *thread safe* [33],

which allows developers of applications based on Kafka to “manually” create their own such constructs. While software libraries have been created with the goal of easing the development of parallel Kafka consumer applications, no such library exists for Kafka producer applications. For the most part, the information available to developers of such applications is limited to basic code examples showcasing how parts of such applications can be developed. Some of these examples also provide basic design guidelines and recommendations for ensuring optimal program performance. These recommendations are seemingly based on unsubstantiated claims, however, as no data supporting them has been presented.

Because of this, and because one major selling-point of Apache Kafka is its high performance capabilities [31], it appears a worthwhile effort to examine and procure data of the performance benefits, if any, of executing producer applications in parallel, using multiple threads on a single system. Such data could provide insight into if parallel producer applications offer improved performance compared to single-threaded producer applications, the magnitude of those performance improvements, and how these vary for a number of different system configurations and computational tasks. Furthermore, by investigating the factors which become bottlenecks and thereby limit the performance improvements attainable by the utilization of parallelism, future improvement areas in the development of Apache Kafka can possibly also be examined.

4 Methodology

This Section presents the experimental setup used when evaluating the performance of the Kafka producers. Sections 4.1 and 4.2 present the design and motivation behind the evaluations, while Section 4.3 explains how the evaluations were carried out, by presenting information such as the hardware used to run the producer applications.

4.1 Producer Evaluation Scenarios

This Section presents the scenarios in which the performance of Kafka producers has been examined. In order to best examine producer performance and behaviour for varying computational loads, two different computational scenarios related to message production have been constructed.

4.1.1 Scenario One - Binary Data

In the first scenario, the task of the configured producer(s) under test is to simply send binary data, available in a buffer having previously been read from a file, at as high of a rate as possible. The same data set was used in each case, consisting of ten gigabytes of binary data read in from a file into a byte buffer of the same size before beginning the timed evaluations of each producer. However, having data available and ready to be sent to brokers in its current state is arguably not really representative of the typical use-case of Kafka producers, evaluations were also carried out under an additional scenario, in which producers were required to perform computational work before messages could be sent. This additional work could disperse the timings at which the executing threads attempt to send messages, thereby possibly reducing contention for network I/O-resources and, therefore, reducing the possibility of such contention becoming a performance bottleneck.

4.1.2 Scenario Two - Trapezoids

The purpose of this second scenario was to emulate the computational work which is typically carried out by producer applications before sending the resulting data to the brokers. While this computational work may be any type of work, one of the most common use cases is that of data transformation and aggregation. However,

the computational work chosen for this scenario was not transformation nor aggregation of data. Instead, the computational work consisted of computing the actual data which was be sent by the producer(s). Specifically, the calculation chosen was that of the *trapezoidal rule*, used to estimate the integral of a function between two points. Put briefly, the integral of a function is estimated by dividing the area under the function curve into a number of trapezoids. The area of each such trapezoid can then be calculated exactly, and the sum of these areas gives the estimated integral.

Although computing trapezoids is arguably not a representative use case of Kafka producers, estimating integrals in this fashion was selected as the computational work for two reasons. First, designing and implementing a parallel solution for calculating an integral estimation based on trapezoids is rather easy; the trapezoid whose areas are to be calculated can be distributed among the available cores, each one calculating a partial approximation of the integral at hand. These partial approximations can then be combined to produce the full approximation, or, as in this case, each approximation can be sent to the brokers by a producer. As such, these computations would utilize the available hardware resources in an effective manner. They would, using the term in a hand-wavy manner, *scale* well.

In addition to this, using this problem to simulate computational work required to send messages enables a simple way to control the rate of message production achieved by the executing producer(s). The computational effort required for estimating the integral of a given function depends on the number of trapezoids used to estimate the integral: the more trapezoids used, the more accurate the estimate produced. Furthermore, and most importantly, this also enables a way to control the rate at which messages are sent by the executing threads. By combining multiple trapezoid areas into each message, rather than sending a message each time a trapezoid area has been calculated, a set number of trapezoid areas must be calculated before each message is sent. The number of areas to be calculated for each message can then be altered, subsequently changing the rate of message production. As the number of trapezoid areas represented by each message increases, the rate of message production decreases, and vice versa. By utilizing this fact, the performance of a solution using multiple producers and that of one using a single producer can then be examined for a number of different rates of message production. As the rate of message production changes, so does the contention for the producer(s) and the I/O resources used to send messages. Because of this, the solution resulting in the best performance *may* depend on the computational work required to send each message. The different number of trapezoid areas per message used to evaluate the producer profiles were 100, 1000 and 10000. In total, the calculation of one billion trapezoid areas formed the computational basis for this scenario.

In order to carry out this examination, two evaluations were performed for each producer configuration under test. In both cases, the trapezoidal calculations were performed in parallel. However, in one case only a single producer was instantiated and shared among all executing threads. In the other, each thread was provided with its own, private instance of the producer configuration to be tested. As such, the performance improvement gained from performing the trapezoid calculations in parallel was present in both cases, with the differentiating factor being only whether

or not several producers are executing in parallel, for varying rates of message production.

The actual function used to estimate the integral by use of the trapezoidal rule was a simple sinus curve, using the `math.h` header file of the C standard library. The part of the x-axis for which the integral was to be estimated was the range $[0, 1000]$. A total of one billion trapezoids were to be calculated by the executing threads. The implementation of the integral estimation using the trapeziodal rule was based on the implementation provided in [47, Ch. 5.2], specifically the implementation using OpenMP for parallel execution. A pseudocode implementation of this is shown in Listing 4.1. In essence, each thread executes a function, given the left and right endpoints of the interval of the x-axis for which the integral is to be estimated, along with the total number of trapezoids to be calculated, the total number of threads, and the identifier of the current thread, which is an integer in the range $[0, n)$ when executing with n threads. Additionally, the number of areas constituting a message is also provided. Then, each thread can calculate its own, local part of the x-axis which it is responsible for estimating. This partitions the x-axis among the threads, making it possible for the threads to execute their work in parallel.

Listing 4.1: Psuedocode implementation of the parallel trapezoidal rule implementation

```
func estimate_integral(left, right, n_trapezoids, n_threads, thread_id, areas_per_msg):
  trapezoid_base = (right - left) / n_trapezoids
  trapezoids_per_thread = n_trapezoids / n_threads
  my_left= left + thread_id * (trapezoid_base * trapezoids_per_thread)
  my_right= my_left+ trapezoid_base * trapezoids_per_thread

  area_cnt = 0

  // Calculate endpoint values
  my_res = (sin(local_left) + sin(local_right)) / 2.0

  for i in 0..trapezoids_per_thread:
    x = my_left_endpoint + i * trapezoids_base
    my_res += sin(x)
    area_cnt++
    if area_cnt == areas_per_message:
      send_message(res)
      area_cnt = 0
```

4.1.3 Producer Allocation

This section describes how the producer instances were allocated to the threads of execution, in order to ensure the instances were shared or thread-private, respectively.

When evaluating the performance of a shared instance, in order to ensure that the same producer was used by all threads of execution, only a single producer instance was created. Each evaluation-scenario was implemented as a separate function. As such, a pointer to this single instance was passed as an argument to each function

representing a scenario where a shared instance was to be evaluated. As the same function, called with the same arguments, was executed by all threads, each thread was provided with the address of the same producer instance.

Evaluating the performance of private instances was slightly more difficult, as ensuring no instance was used by more than one thread required a little bit of finesse. The approach used was to allocate a matrix of producer instances. One row of producers was initialized for each executing thread. Each column in this matrix corresponded to a producer configuration profile. In order for the threads to know which row to access, the function `omp_get_thread_num()`, was used. This function returns a unique identification number of the calling thread, allowing each thread to use it as a mechanism for identifying themselves, and their producer instances. As in the case of shared instances, the evaluations were implemented as functions, but instead of passing a pointer to a single producer instance, a pointer to the matrix was passed when calling the functions. As such, each thread could use its identification number to access its row of producer instances, ensuring no instance was used by more than one thread.

4.2 Kafka Producer Configurations

In order to gain a thorough understanding of the performance effects of running producers in parallel, a range of producer configurations were created and put under test. By doing so, the performance of producers executed in parallel could be examined for multiple possible application requirements, in order to determine how these requirements might affect any performance gains.

In order to keep the number of examined configurations within a reasonable limit, only the configuration parameters deemed to have the largest effect on producer throughput were altered when the producer profiles were constructed. These configuration parameters are message acknowledgement, whether EOS is enabled, message compression, and message batching, the details of which are presented in Sections 2.3.2, 2.3.3, 2.3.4, and 2.3.5, respectively.

Three producer profiles were created for the primary performance evaluations. The configured parameter values of these profiles are presented in Table 2. Any parameter not explicitly shown in the table is assigned the default value, as shown in [9]. The table only contains configuration parameters related to message batching and

Table 2 The three primary producer configurations to be evaluated

Producer Profile	Linger Time	Batch Size	Queue Buffer Size	Compression
Basic	Default	Default	Default	Gzip
Medium	Medium	Medium	Medium	Gzip
Large	High	High	High	Gzip

compression. The reason for this is that each profile in the table was evaluated twice for each evaluation scenario, once without any acknowledgement messages nor EOS,

and once with acks set to all and EOS enabled. Most of these configuration options have a large range of possible numerical values, far more than is reasonable to cover exhaustively in performance tests. As such, three distinct levels were created in order to cover values at different points in these numerical ranges. These three value are: *Default*; *Medium*; and *Maximum*.

The Default values are, as the name suggests, the values assigned by the default producer configuration. Both Medium and High represent values which are larger than those represented by Default. This is because most producer configuration guidelines recommend increasing the values of the configuration options in Table 2 in order to achieve a higher throughput [55, p.54-61]. The Medium level represents values which are larger than the default ones, while still being quite far from the allowable maximum. The values represented by the Maximum level are, fittingly enough, the maximum allowable values. By evaluating producers using both Medium and Maximum values, some variability in the selected configuration values is achieved, while still following the general configuration guidelines for producers. For the exact values represented by these levels for each of the configuration options, see Appendix A.

All profiles made use of message compression, using the same compression algorithm, *gzip*. The reason for always using compression is that doing so tends to increase the rate of data throughput [27]. The specific choice of gzip as the compression algorithm was based on the fact that the CPU on the system running the producer applications was fairly powerful. As such, the trade-off of increased CPU usage for reduced network bandwidth usage seemed reasonable.

For each profile, the evaluations carried out were the same; in the first case sending binary data without any additional computational work, and in the second case carrying out some computational work before sending the produced result to the broker.

4.3 Experimental Setup

This Section presents how the performance measurements for the various producer configurations were carried out.

Two separate computer systems were used to carry out the performance measurements. One system ran the application with the Kafka producers, along with the data and computation representing messages they were to send. The other system was used to host the broker which was responsible for receiving the messages sent by the producers, as well as the Apache ZooKeeper server, responsible for coordinating brokers. These two systems were both connected to the same local area network, located within a few meters of one another. The system hosting the brokers used a wireless connection, using the 2.4 GHz Wi-Fi radio band, while the system running the producer applications used a wired connection. The wireless connection had to be used due to hardware constraints, i.e. lack of an Ethernet port on the system. By running the Kafka broker on a separate system, rather than on the same system

as the producers, more realistic conditions could be achieved, as the messages were sent across a computer network, which would not be the case if the messages were to be sent and received by the same system. However, it should be noted that while the networking conditions under which the evaluations were performed are realistic, they are also close to optimal, as the distance between the involved parties was very short, and the amount of traffic on the network beyond the data sent via Kafka was kept to a minimum. It is easy to imagine a scenario in which the system at hand is distributed on a larger scale, with the producers and brokers being hosted on separate networks, with large amounts of additional data being sent via the same network.

The hardware used by the system running the producers was an AMD Ryzen 5 3600x 6-core processor. In the case of this processor, each such core has a total of two hardware threads, by the use of the use of *Simultaneous Multithreading* [30], extending the total number of hardware threads to twelve, running at a base clock frequency of 3.8 GHz with a maximum frequency of 4.4 GHz. This system had a total of 32GB of *Random Access Memory (RAM)*, running at a frequency of 3200 MHz.

The system hosting the brokers used an 8-core Intel Core i7-10510U processor, running at a base clock of 1.8 GHz, with a maximum clock frequency of 4.9 GHz, with 16 GB of RAM. In order to measure how the performance of the producers under test varied with the number of threads used to execute the program, each parallel evaluation was run using 2, 6, and 10 threads. Additionally, in order to measure the achieved speedup and efficiency, a sequential implementation running on a single thread was also used to evaluate each of the producer profiles in both scenarios.

The source code for the evaluation program can be found at [57]. In order to produce measurements which were representative for the average-case performance of the systems under test, each evaluation was run for a total of five times. For each run, the elapsed time was measured using the OpenMP function `omp_get_wtime`, which can be used to measure *wall time*, i.e. the actual time elapsed between the measurements, not the time spent in the program by the processor. The wall times of each of the five measurements was summed up and then averaged. An increased number of runs would likely produce a more accurate average-case measurement. However, as some of the measurements required a rather significant running time, the number of measurements was limited to five per system configuration.

Beyond elapsed time, a number of other statistics related to the Kafka producers were also measured. These include a wide range of different measurements, including several types of delays at different points in the message-sending process. These were among the most interesting such statistics, as they revealed at which stages in the process messages were delayed the most, as well as how message latency was affected by the various configurations. These measurements were recorded by using a special statistics callback function, which recorded measurements in the format of a *Javascript Object Notation (JSON)* file. This callback function was configured to be executed once every two seconds, frequent enough to provide a sufficient number of measurements, but not frequent enough as to have a significant impact on overall performance.

In order to ensure that the performance measurements were as representative as possible, the span of time that was measured was kept as limited as possible, to only measure the time required for sending messages as well as that required for computing the trapezoids. As such, any set-up and tear-down, such as creating and destroying producers, or loading binary data from file, was not included in the wall-time measurements. Furthermore, a waiting period of twenty seconds between each measurement was established. The reason for this was to ensure that the broker receiving the messages would be able to process all messages at a high rate, minimizing the possibility of the broker becoming a performance bottleneck.

Finally, in order to ensure maximum program performance, the evaluation application was compiled with the `-Ofast` compiler flag [36], which results in the compiler disregarding strict adherence to the language standard in order to optimize the code to the furthest possible extent.

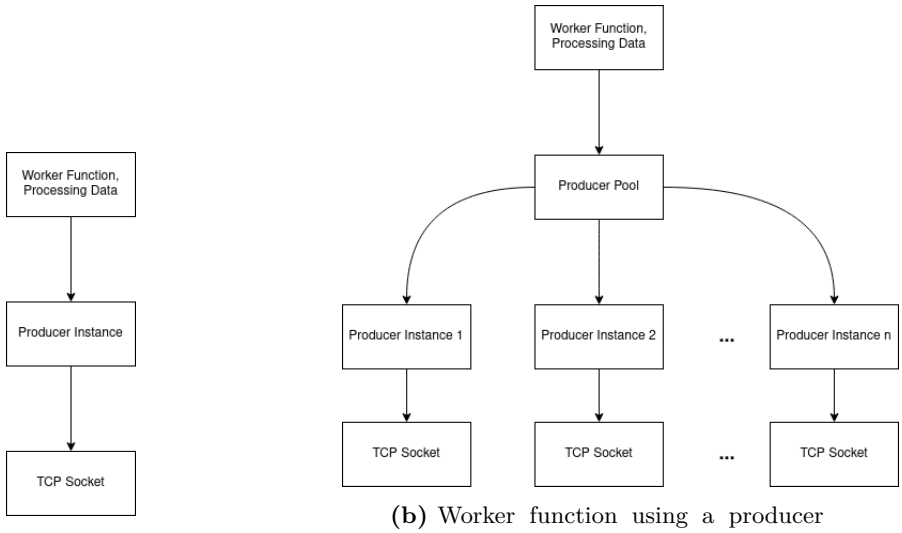
4.4 Design of the Software Library

This section presents an overview the chosen design of the constructed software library, as well as the motivations behind the design choices made. The intended use-case of the library is also presented.

The starting point for the design of the library was the way in which Kafka producer applications are typically designed, regardless of whether they are executed sequentially or in parallel. An aspect of such applications which is fairly common is how work is carried out and producer instances used to send messages. Typically, the work required to create the messages to be sent by the producer is carried out in some form of loop. This loop is, in turn, typically placed within some form of function, which is called with the producer instance and data to be processed. It is this construct, of a "worker function" which processes data and produces messages that the design of the library is centered around, and aims to keep intact. By doing so, the development process of parallel producer applications based on the library can be kept as simple and as similar to the current process as possible.

During the evaluation of possible design choices for the library, two possible approaches were identified. The first of these aimed to increase the performance of already existing worker functions which make use of parallel execution but rely on only a single producer instance. The aim of the second was to allow existing sequential worker functions to be used without the need for any alterations, by passing each worker function at hand as an argument to a construct defined by the library. In turn, this construct, referred to as a "wrapper function" would be tasked with executing the given worker function in parallel.

An overview of the first of these approaches can be seen in the diagram shown in Figure 5b, along with a diagram over the more conventional use of a worker function, shown in Figure 5a. As shown in Figure 5b, this approach is based on replacing the single producer instance used in worker functions with a new construct, called



(a) Conventional worker function, using a single producer instance

(b) Worker function using a producer pool, consisting of multiple producer instances (logic not shown)

Figure 5: Overview of a conventional worker function and one using a producer pool

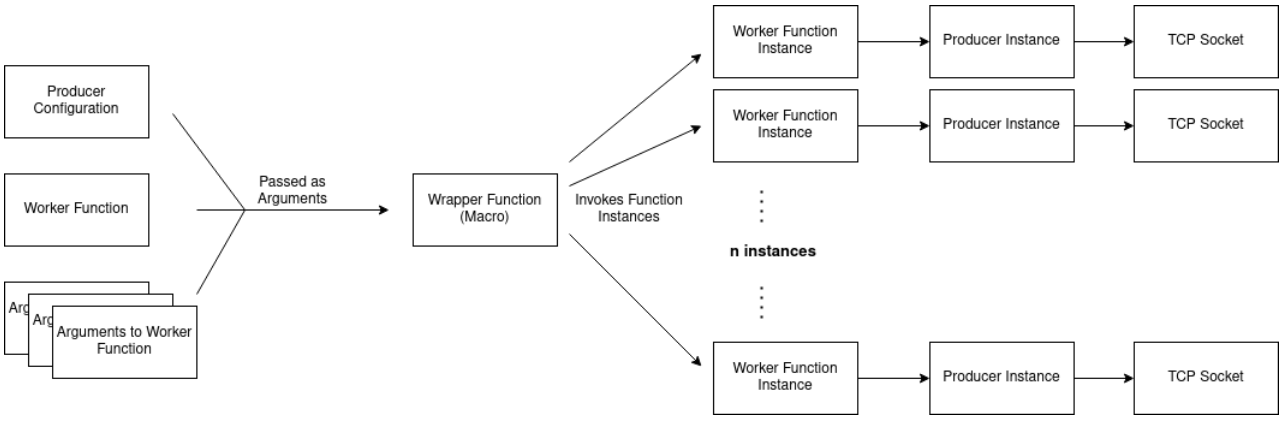


Figure 6: An overview of usage of the wrapper-function exposed by the library

a `ProducerPool`. This pool is essentially a collection of multiple producer instances combined with logic for varying which producer in the collection is used between each message sent. Distributing the messages to be sent among multiple producer instances in this manner was seen as a possible way to avoid a performance bottleneck caused by the performance limitations of an individual producer instance. Additionally, although the use of such a pool would require altering the source code of the given producer application, these alterations would be minor ones, essentially limited to instantiating correctly configured pools and substituting these for the producer instance currently used.

A diagram showing the design and intended use-case of the second approach identified in the early stages of the design process can be seen in Figure 6. In this setting, the worker function, implemented sequentially and making use of a single producer instance, is passed to the wrapper function exposed by the library. As such, no

modifications to the worker function itself are needed. Instead, an invocation of the exposed wrapper function needs to be added, with the worker function and producer configuration as arguments. Additionally, the arguments expected by the passed worker function also need to be supplied. Here, one point of potential complexity arises. During the initial design process, the only identified way to execute the passed worker function in parallel was to invoke the worker function multiple times, each invocation being made by a separate thread. However, executing multiple instances of the worker function with each instance working on the same data may not always be particularly useful, as it may lead to the same work being executed multiple times. As such, when passing the arguments intended for the worker function as arguments to the wrapper function, multiple sets of such arguments may be supplied. Each such set will then be used in separate invocations of the worker function, which makes it possible for each instance of the worker function to work on separate data.

One final hurdle of note identified during the initial design of the library was how to enable the exposed wrapper function to accept worker functions without knowing the number and types of arguments expected by the worker function. Because these arguments are passed to the wrapper function, this would affect the arguments expected by the wrapper function, as well, and because this function is exposed by the library, users of the library should not be required to alter the library source code in order to be able to use it. As the library was to be developed using the Rust programming language, it was decided that part of the library was to be implemented using Rust's advanced macro system. In effect, this lead to an implementation where separate source-code implementations of the wrapper function are generated at compile-time, based on the number and types of arguments to be passed to the worker function. As a technical detail, this meant that the wrapper function exposed by the library is in fact a macro, not a function.

Which of the two constructs provided by the library is the better choice largely depends on the requirements of the producer application at hand. More data regarding the performance of these constructs can be found in Appendix B.

5 Results

This Section presents and summarizes the performance data resulting from the profile evaluations. First, the results from the binary-data scenario are presented for each of the profiles and message sizes. Finally, the data from the scenario where trapezoid areas are first calculated and then sent to the broker is presented.

5.1 Execution Times, Shared and Private

This Section presents data related to the execution times measured in the two experimental scenarios, with a particular focus on how the execution times are affected by whether a shared producer instance or private instances are used.

The execution times in the binary data scenario, averaged over all producer profiles, are visualized as a bar graph in Figure 7, and are shown numerically in Table 3. In the table and the graph, a distinction is also made between whether a single producer instance is shared between all executing threads, or if each thread is assigned its own, private producer instance. Furthermore, the achieved speedup and efficiency values are also presented in the table.

Table 3 Average wall times, speedups and efficiency values

Threads	Producer configuration	Wall Time (s)	Speedup	Efficiency
1	Not applicable	27.91	Not applicable	Not applicable
2	Private	10.42	2.68	1.34
2	Shared	31.20	0.89	0.45
6	Private	9.10	3.07	0.51
6	Shared	61.34	0.46	0.08
12	Private	4.85	5.75	0.48
12	Shared	31.44	0.89	0.07

As can be seen by the data presented in the graph and table, when executing the binary data scenario, the execution time is only reduced, compared to the execution time achieved by a single thread, when private producers are utilized. In this scenario, all cases where a single producer is shared between executing threads results in an execution time *longer* than that achieved when executing using a single thread. Additionally, as might be expected, increasing the number of cores used for execution reduces the execution time, as long as private producers are used. However, as the number of cores is increased, the efficiency of each core decreases, along with the increased speedup value achieved by the additional cores.

In the second scenario, where a number of trapezoid areas are to be calculated before

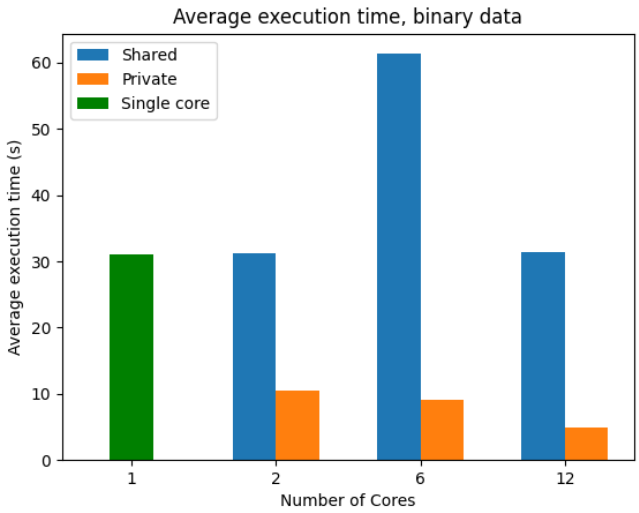


Figure 7: Execution time, averaged over all producer profiles, on binary data

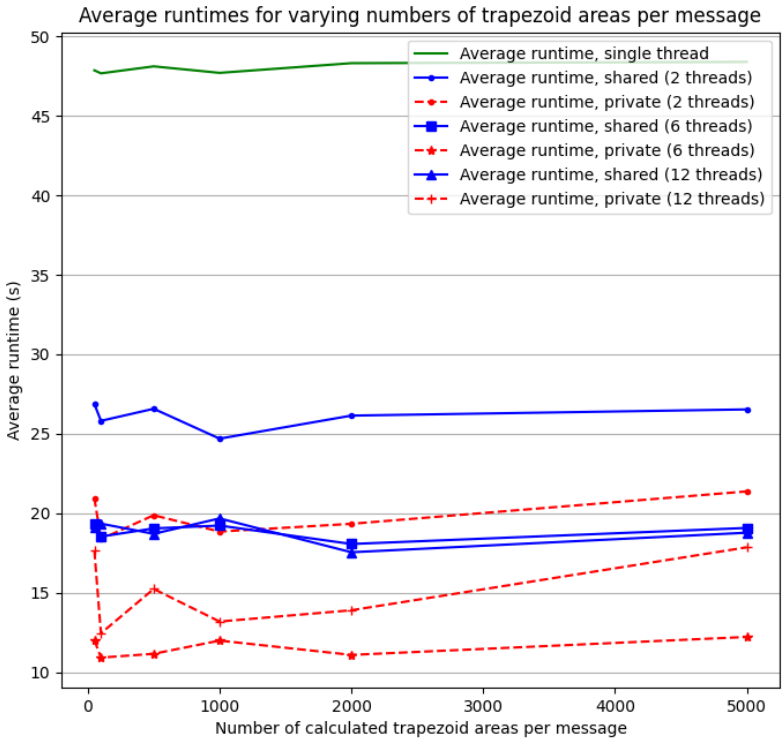


Figure 8: Execution times, averaged over all producer profiles.

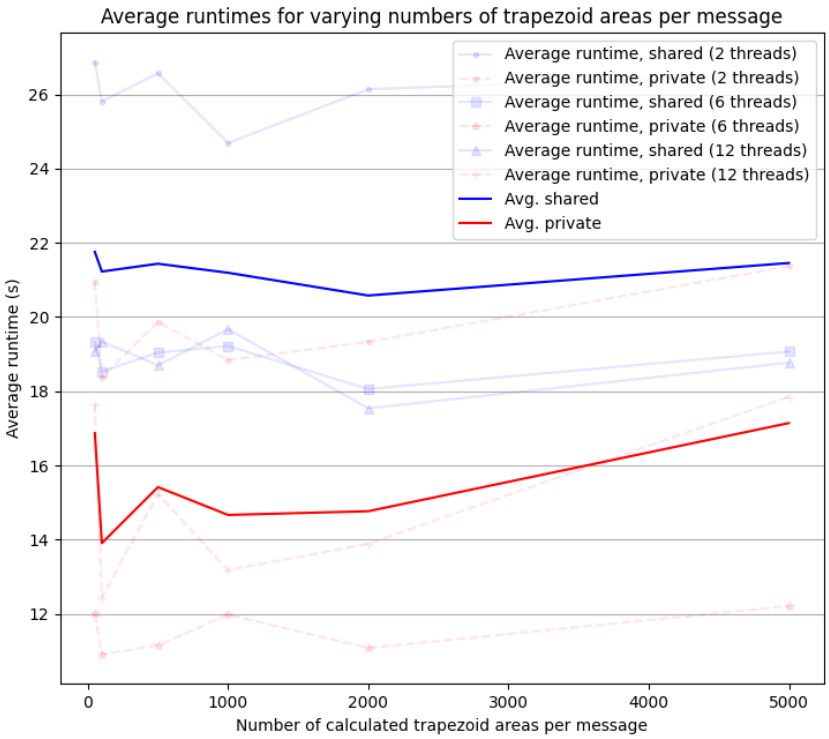


Figure 9: Average execution times for shared and private producers.

each message is sent, the results are similar. The execution time, averaged over all producer profiles, for varying numbers of executing threads and areas per message is shown in Figure 8. Figure 9 visualizes the same data, with the addition of highlighting the average execution time of shared and private producers, averaged over the number of executing threads.

As can be seen in the figures, using private producer instances does, on average, result in shorter execution times than utilizing shared producers. In fact, for any set number of executing threads, utilizing private producers results in a shorter execution time than using a shared producer. In all cases, using multiple threads of execution results in a shorter execution time compared to the time required by sequential execution. However, as the number of trapezoid areas to be calculated is the same in all cases, the number of areas each thread has to calculate decreases as the number of threads is increased. As such, this reduction in execution time is arguably to be expected, and is likely not greatly influenced by how the producer(s) are utilized to send messages, nor their configuration.

Additionally, how the number of calculated areas per message affects producer performance may also be of interest. As can be seen in the two Figures, as the number of areas per message increases, so does the execution time. Additionally, as this measure increases, the performance difference between shared and private producers seems to decrease somewhat. At the lower end of this scale, there is more variation in the measured execution times. Overall, calculating 50 areas per message seems to result in the longest execution time, while 100 areas per message results in the shortest one.

Examining the number of executing threads, it should be noted that in the case of private producers, executing 6 threads always results in shorter execution times than doing so with 12 threads, even though the latter setup results in half as many trapezoid areas needing to be calculated per thread.

As the execution times in the trapezoid scenario are affected significantly by the number of areas each thread has to calculate, and are therefore affected by the number of executing threads, the absolute values of these execution times may not be of that much interest when trying to examine the performance effects caused by using shared or private producers. Instead, one metric that may be of more interest is the performance difference between a shared producer and private ones, for a fixed number of threads, and how this difference is affected by the number of areas per message.

Figure 10 contains a graph which visualizes the difference in performance when utilizing shared and private producers for a number of different areas per message and number of executing threads. The values on the x-axis represent the number of areas calculated for each message sent. The y-axis shows the ratio $\frac{shared_runtime}{private_runtime}$, i.e. the measured execution time when utilizing a shared producer, divided by the measured execution time when utilizing multiple private producers. This measurement is shown for different numbers of executing threads, where the lowest value on the y-axis (i.e., the x-axis), is 1.0.

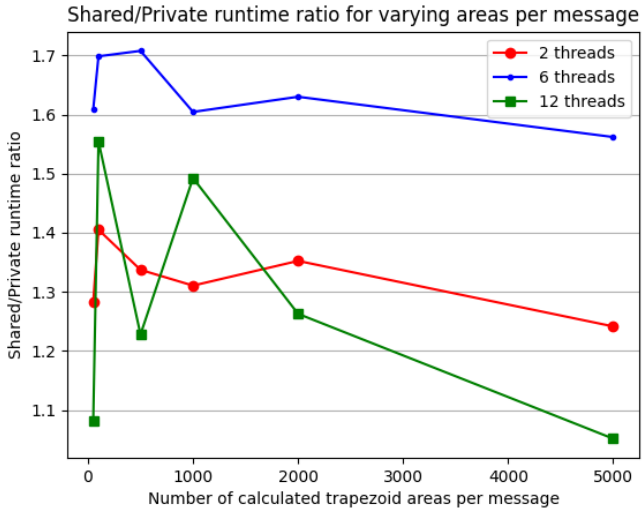


Figure 10: The ratio between the measured execution time using a shared producer and the execution time measured with private producers.

Table 4 Predicted and Actual average execution times, trapezoids

Number of Cores	Predicted Time	Actual Time	Prediction/Actual
2	23.5	19.83	1.19
6	7.83	11.51	0.68
12	3.92	15.1	0.26

As can be seen in this Figure, the value of this ratio is always greater than 1.0, meaning that it is faster, for all measured areas per message, to use thread-private producers compared to a single shared one. Trend-wise, the value of this ratio seems to decrease as the number of areas per message is increased to 5000. For the other values of areas per message, there is a lot more variation in the ratio value. Additionally, the largest value of the ratio is achieved when executing with 6 threads, which may be expected as this thread count resulted in the best execution time with private producers.

The execution time for each thread count, averaged over the number of areas calculated per message, can be calculated from the individual points shown in Figure 8. These averages, along with the average execution times as predicted by the model, can be seen in Table 4.

5.2 Message Latencies

While the achieved execution times may present how well certain configurations perform compared to one another, further insight may be attained by examining how long some parts of the message production process take. For example, in the binary

data scenario, the performance improvements realized by increasing the number of threads seems to stagnate as the number of cores increases. Similarly, in the trapezoid scenario, the execution time increases as the number of threads is increased from 6 to 12. Insight into why this is the case, such as possible bottlenecks, can be provided by measuring the time spent in the various stages of the message production process.

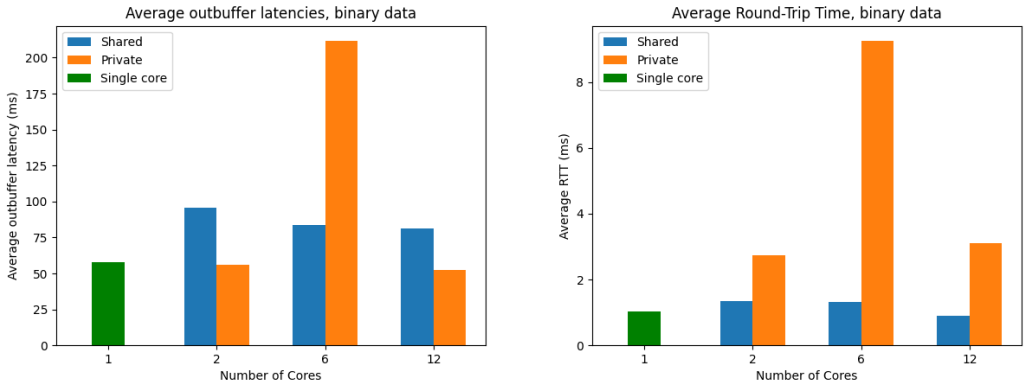
Two such time measures are the *outbuffer* latency, which is the duration between the time when a send request is enqueued on the producer output queue and the time this request is written to a *Transmission Control Protocol (TCP)* socket, and the *internal* latency, which is the time spent by messages in the producers internal message queue.

Figure 11a presents the outbuffer latencies, averaged over all producer profiles, for varying numbers of executing threads, measured on binary data. As can be seen in the Figure, in most cases, utilizing private producers results in a lower outbuffer latency, meaning message batches are written to a TCP socket, and therefore sent to brokers, more quickly. The exception to this seems to be when 6 cores are used for executing the program, for which the measured outbuffer latency increases significantly. It is unclear exactly why this occurs, and it is arguably a reasonable assumption that it is due to some outlier values caused by some network anomalies. Exactly why this occurs is not clear. However, some insight may be given by Figure 11b. The Figure presents the measured *Round-Trip-Time (RTT)* between the producer and broker, i.e. the time required for a request from the producer to be received by the broker and the response from the broker to be received by the producer. High RTT values may be an indicator of network congestion [6]. As such, the high RTT values, and the underlying network contention they indicate, measured for 6 private producers may be a possible explanation for the large outbuffer latency measured with the same configuration.

Additionally, for all thread counts in the binary data scenario, utilizing a single shared producer results in a longer outbuffer latency than when executing with a single thread and producer.

The total amount of time, on average, spent by messages in the internal message queue of a producer, when sending messages consisting of binary data, is shown in Figure 12. As can be seen in the Figure, utilizing private producers on binary data results in messages spending significantly less time in the message queue of producers, compared to when a shared producer or a single producer, executing sequentially, is used. Furthermore, in the case of private producers, the internal producer latency seems to decrease steadily as the number of executing threads increases.

On the other hand, when sharing a producer instance among the executing threads, the internal producer latency decreases only slightly, compared to a producer executing sequentially. Additionally, no clear trend indicating that this latency is reduced as the number of executing threads increases seems to exist. Overall, the internal producer latency of a shared producer instance seems to be about on par with a sequentially executing producer, regardless of the number of executing threads.



(a) Outbuffer latency, using a varying number of threads, when executing on binary data

(b) Average Round-Trip-Time between the producer system and broker host, when executing on binary data

Figure 11: Outbuffer latency and Round-Trip-Time measured on binary data

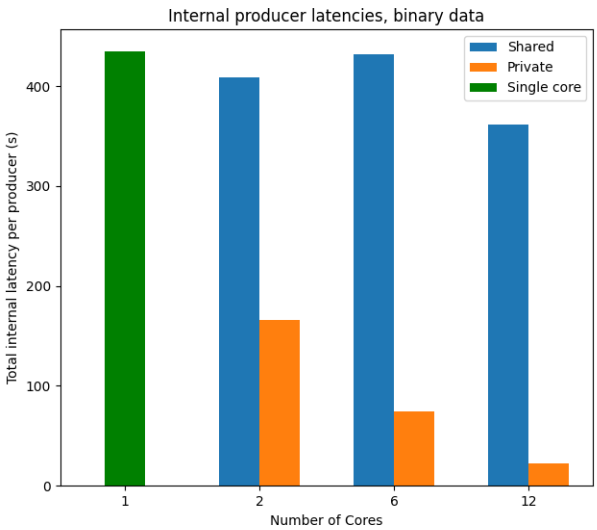


Figure 12: Internal producer queue latency, using a varying number of threads, executing on binary data

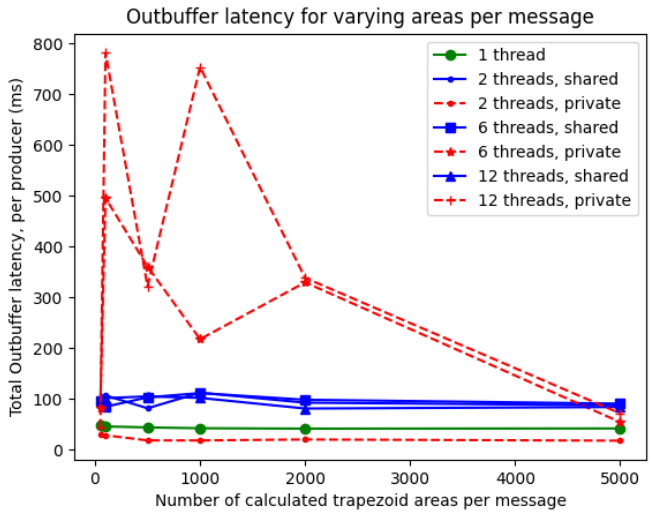


Figure 13: Measured outbuffer latency for varying thread counts and areas per message

Just as for the scenario where binary data is sent, the amount of time elapsed in different parts of the message sending process may be of interest when calculating trapezoid areas. The outbuffer latency for shared and private producers, for different numbers of executing threads and as the number of areas per message is varied, is shown in Figure 13. A somewhat simplified view of this data, which highlights the averages of all thread counts for shared and private producers, respectively, is shown in Figure 14.

As can be seen in these Figures, the outbuffer latency is, on average, longer when utilizing a private producer per thread compared to when using a single shared producer. The only exception to this trend occurs when only two threads are executing, each with its own producer. Furthermore, it seems to be the case that in general, and for private producers especially, larger outbuffer latencies occur as the number of areas per messages decreases, i.e. as the rate of message production increases. In addition to this, increasing the number of executing threads seems to increase the outbuffer latency of producers.

The time spent by messages in the internal producer queue is shown in Figure 15, with the averages of shared and private producers shown and highlighted in Figure 16. As can be seen in these Figures, the time messages spent in the internal producer queue is significantly longer when using a shared producer instance, compared with multiple private producer instances. Furthermore, the internal latency of shared producers is greater than that which occurs when producing messages sequentially with a single producer, for all measured number of areas per message.

Looking at how this latency is affected by the number of executing threads, Figure 13 shows that the private producers and a shared instance area affected differently by the thread count. For private producers, increasing the thread count seems to reduce the internal producer latency, as the longest such latency is achieved when

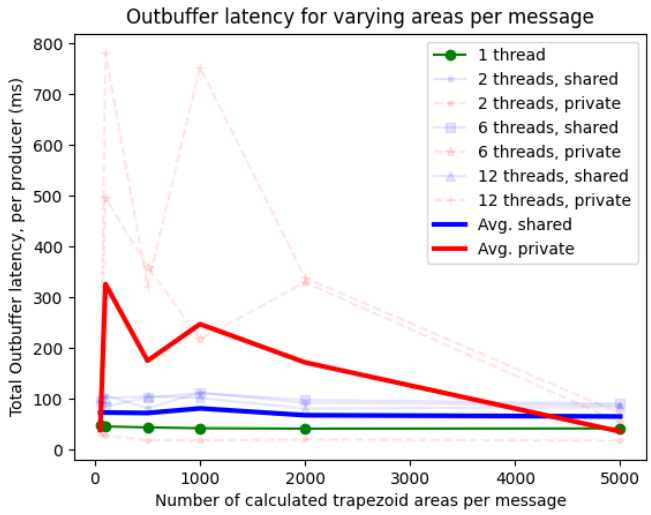


Figure 14: Average outbuffer latency for shared and private producers, as the areas per message is varied

executing using 2 cores, with 6 and 12 cores resulting in somewhat shorter latencies. On the other hand, when utilizing a single shared producer instance, increasing the thread count seems to lead to an increase in internal producer latency, as the shortest latency is achieved when two threads of execution are used, with both 6 and 12 threads resulting in longer latencies.

5.3 Configuration-specific Results

An additional aspect which may be worthwhile to consider is whether the performance difference between shared and private producers is affected by the configuration used when creating the producer instance(s). Depending on the requirements of the application being developed, certain restrictions on the behavior of producers, in regards to aspects like reliability, may exist. As such, it may be of interest to examine if such restrictions could potentially limit the performance improvement gained by parallel execution.

One argument against allocating a producer instance to each thread of execution is that the increased network overhead worsens the overall performance, compared to when a single producer instance is used. Furthermore, the network overhead of producers generally increase if they are configured for reliability, i.e. have to wait for acknowledgement messages that are sent across the network, and/or use EOS. As such, it may arguably be reasonable to expect that the performance achieved when using private producers, compared to that achieved when using a shared producer, should be affected by the producer configuration, specifically regarding whether acknowledgement messages are used.

Figures 17, 18 and 19 contain graphs which show the execution times of three pro-

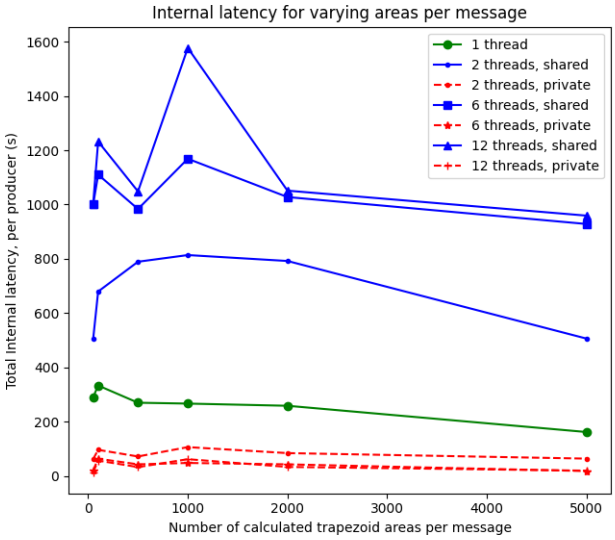


Figure 15: Total internal producer queue latency per producer

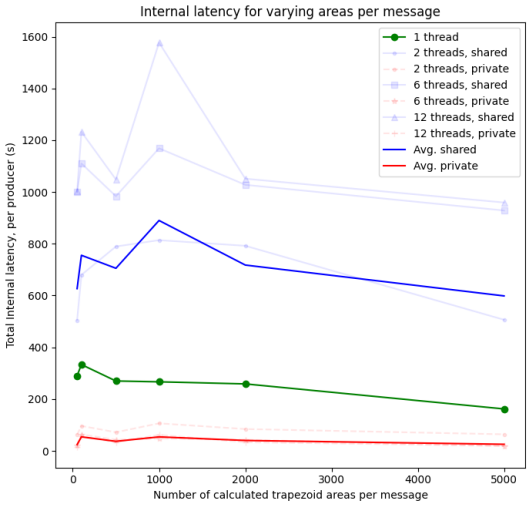


Figure 16: Average internal producer queue latency, for shared and private producers

ducer profiles. The execution times are the average execution times when executing the binary data scenario, as well as the trapezoid area scenario, for which the shown execution times are the average over all areas per message. Each scenario is shown in its own graph. The profiles are the three profiles presented in Section 4.2: a *basic* producer, with no configuration other than the default, a *medium* producer, configured according to guidelines for increased throughput, by increasing the values of certain configuration parameters, and finally a *large* producer, which increases these values even further. Each profile has been evaluated four times: without acknowledgement messages and without EOS, and with these settings enabled, in both a shared and private setup. The increased execution time incurred by enabling acknowledgement messages and EOS is shown as the hatched parts of the bars.

As can be seen in these Figures, it is the case, for all number of cores and profiles, that using private producer instances results in faster execution times, on average. Furthermore, enabling acknowledgement messages and EOS increases the execution time in all cases. However, the magnitude of this increase is not always the same. For example, it seems as though this increase is generally larger for a single shared producer instance than it is when utilizing private instances. Furthermore, the relative magnitude of this increase seems to decrease as the overall execution time increases, which can be seen by noting that the basic producer configuration results in the shortest execution times, the medium one slightly longer times, and the large profile even longer ones. This means that the magnitude of the overhead in absolute terms remains largely stable, regardless of the producer configuration used.

A comparison of the data shown in the trapezoid graphs with that shown in the binary data ones reveals that the performance difference between shared and private producer instances tends to be significantly larger in the binary scenario. Additionally, in the binary scenario, this difference seemingly increases as the number of thread counts increases. On the other hand, in the trapezoid scenario, using 12 threads of execution results in rather similar execution times, regardless of whether acks and EOS is enabled or not. This is the case, in particular, for the basic producer configuration.

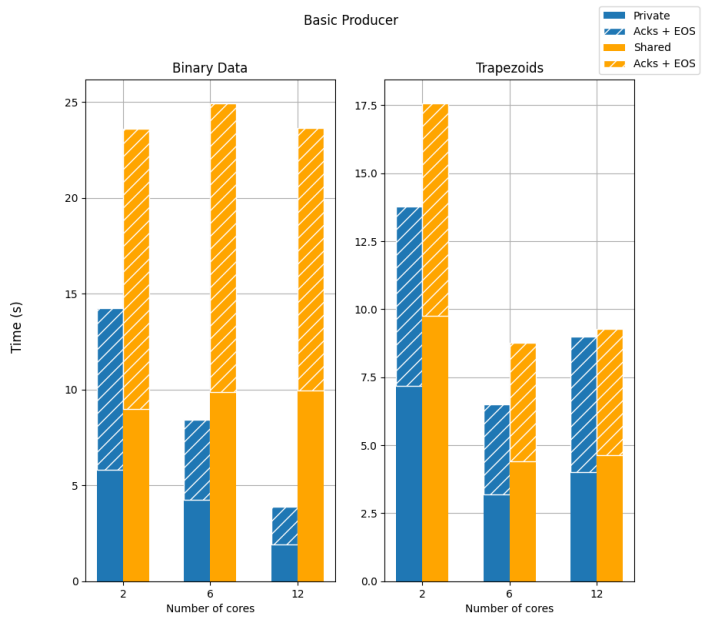


Figure 17: Execution time comparison with and without acknowledgement messages and EOS, basic producer configuration

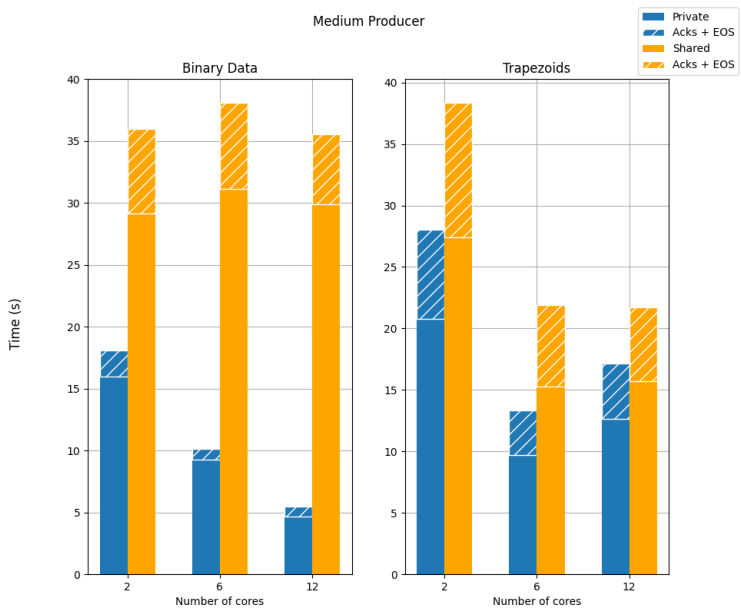


Figure 18: Execution time comparison with and without acknowledgement messages and EOS, medium producer configuration

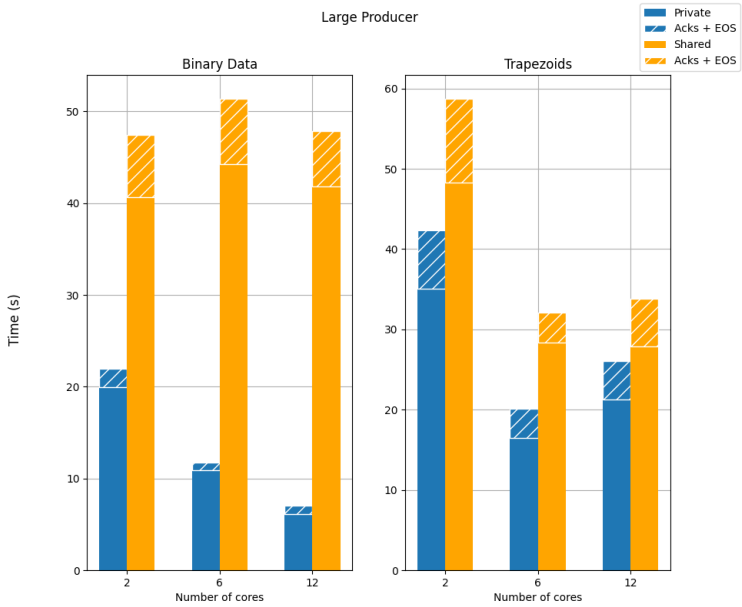


Figure 19: Execution time comparison with and without acknowledgement messages and EOS, large producer configuration

6 Discussion

This Section presents a discussion surrounding the results presented in Section 6. Specifically, the throughput, through the measured execution times, the measured internal and outbuffer latencies, and the effect caused by the configuration used to instantiate the producer(s) are discussed.

Throughput. From the execution time data presented in Figures 7, 8, 9, and 10, it is arguably rather clear that overall, allocating a separate producer instance for each thread of execution results in significantly better performance, in terms of data throughput, than each such thread sharing a single producer instance. However, as can be seen in Figures 9 and 10, the difference in performance which results from whether a shared instance or private instances are used seemingly decreases as the rate of message production decreases. This is arguably to be expected; as the rate of message production decreases, so does the extent to which the producers are used in the application, thereby reducing their possible impact on overall application performance.

Another point of interest that is likely related to this fact is the significant change in the performance difference between shared and private producers when comparing the binary scenario with the trapezoid one. In the binary scenario, utilizing a private producer instance per executing thread results in significant performance gains compared to utilizing a shared instance. On the other hand, this difference seems to decrease in the trapezoid scenario. This is particularly true for the execution times measured with 12 threads of execution, which are rather similar, especially when the basic producer profile is used. The primary reason for this is likely that the data shown in the trapezoid scenario graphs is averaged over all area counts per message, i.e. over varying rates of message production. As previously explained, the difference between shared and private instances seemingly decreases as the rate of message production is decreased. This rate is at its highest in the binary data scenario, where no additional computation needs to take place before messages can be sent. As such, it is to be expected that this scenario also results in the largest performance differences.

Comparing the results with the theoretical, *queue-based model* as presented by Equation 2.1 and the definitions preceding it, several points can be noted. Firstly, according to the model, increasing the number of publishers by a factor of n also increases the average processing rate by the same factor, $n \cdot \mu$, as each publisher is assumed to be able to process requests at the same rate. In turn, as the utilization of the system, which only denotes the fraction of time the system is in use, is assumed to be unchanged, this increase in average processing rate results in an equal decrease in both the active processing time τ and the average latency L.

Looking at the execution times for binary data presented in Figure 7, we can see that this is **never** truly the case. Increasing the number of threads from one to two results in an execution time which is roughly 0.33 of the sequential execution time, though the processing rate was only increased to 2μ according to the model. As such, the performance improvement in this case is greater than that predicted by the model. Further increases, to 6 and 12 threads, result in execution times ≈ 0.267 and ≈ 0.133 of the sequential one, respectively. According to the model, these reductions should be ≈ 0.167 and ≈ 0.083, corresponding to processing rates of 6μ and 12μ, meaning that in this case, the model overestimates the performance improvements caused by increasing the number of producers.

The values shown in Table 4 present a situation similar to the situation for the binary data scenario; when using two cores, the execution time is shorter than the one predicted by the model, while the opposite is true for the values corresponding to six and twelve cores.

The fact that the execution times predicted by the queueing-theory model *do not align* with the measured execution times, and that the magnitude of this discrepancy increases as the number of cores increases, indicates that the model does not really conform to how the producer actually works. This is arguably not very surprising, as the model does not take the surrounding environment of the modeled process into account, and is therefore *too simple*. This simplicity may also explain why the discrepancy between the measured and predicted execution times increases as the number of cores and producer instances increases; as these increase, so does their contention for computational resources, such as access to network I/O, as well as the overhead incurred by the operating having to handle additional threads. As the model does not take the surrounding environment into account, neither the increased contention nor the overhead are taken into consideration when predicting execution times, resulting in overly *optimistic predictions*.

Latencies. Looking at the measured latencies, in both the binary and trapezoid scenario, using a shared producer instance results in a significantly longer internal latency than that measured when shared instances are used. On the other hand, in most cases, using private instances results in a larger outbuffer latency than a shared instance. This is particularly true when both the rate of message production is high, and the number of executing threads is large. The reason for this behaviour is likely that with a high rate of message production and a large number of threads, the contention for network I/O is at its highest, caused by the large number of threads frequently trying to gain access to it. As such, the threads each have to wait in order to write their message batches to a TCP socket, and they effectively throttle each other. Keep in mind that the performance data presented in this work were measured under very good network conditions, meaning it is arguably rather likely that the effects of this situation are only amplified under poorer network conditions. However, it is important to note that there is a large difference in the magnitude of the measured internal and outbuffer latencies, as the former is measured in seconds, and the latter in milliseconds. As such, the achieved reduction in internal latency is likely to outweigh any increase in outbuffer latency caused by the use of private

producers.

Configuration. In terms of the effects on performance caused by the producer configuration, one significant result shown in Figures 17, 18 and 19 is that the overhead, in the form of longer execution time, caused by enabling acknowledgement messages and EOS is, in some cases, significantly larger when a producer instance is shared among the threads of execution, compared to when each thread is assigned its own private instance. This difference seems to be most significant when the rate of message production is high, as it is in the binary data scenario. One explanation for this could be that when only a single producer instance is used, it is solely responsible for both sending messages and waiting for the corresponding acknowledgement messages, for all executing threads. As such, even though there may be multiple threads with data ready to be sent, none of them will be allowed to send any data if the producer is waiting for an acknowledgement message for the data sent by some thread. This would likely cause a significant bottleneck, limiting the achievable throughput of data. On the other hand, when multiple, private instances are used, message production and waiting for acknowledgement messages is done independently for each thread, removing this bottleneck. One fact which could be used to argue against this is the fact that the *acks* + EOS overhead seemingly does not increase as the number of executing threads increases, which seems like it would be a natural consequence of additional threads contending for producer access, increasing this bottleneck effect. However, the opposite does seem to occur when private instances are used, namely that the overhead seems to decrease as the number of threads increases. Overall, the cause of this discrepancy in overhead is not entirely clear.

Examining the effects of the other configuration options, namely those regarding message batching, one of the most interesting results shown in Figures 17, 18, and 19 is that the basic producer configuration results in the shortest execution time, and thereby the highest throughput, followed by the medium producer and lastly the large configuration. As the medium and large configuration profiles were created in an attempt to adhere to recommendations for increasing producer throughput, this result is arguably somewhat surprising. However, one explanation may be found by considering the fact that the aim of message batching is to reduce the network overhead incurred for each sent message, and that this reduction may require a trade-off, such as additional computation. As the network conditions of the tests were very good, it may be that this trade-off simply is not worth it, resulting in a lower throughput.

In summary - to answer the research question posed in the introduction of this thesis - providing each executing thread a separate, private producer instance *does* result in better performance, in terms of throughput rate, than sharing a single instance among the threads of execution. The extent of this performance improvement depends on several factors, such as the number of threads used, the rate of message production and the configuration used to instantiate the producers. The largest reason for the performance improvement seems to be a significant reduction of the time spent by each message in the internal queue of the producer instance(s). A possible bottleneck, which may prevent the performance to improve further as the

number of threads are increased is an increase in the time the producer(s) need to wait before being able to write the messages to a TCP socket.

7 Conclusions and Future Work

This Section summarizes some conclusions that may be drawn from the results presented in this work. Furthermore, some examples of possible future work, which may improve and expand upon this work, are also presented.

7.1 Conclusions

Overall, taking into consideration the results presented in Chapter 6 and the discussion surrounding these in Chapter 7, it seems that a rather strong argument can be made for utilizing private producer instances instead of a shared instance. Specifically, this seems to result in significantly higher data throughput rates, at least under favourable network conditions. As such, if the application requirements and use-case, as well as the development resources at hand allow it, utilizing private instances seems to be the better option.

This contradicts the Kafka producer documentation, which states that using multiple instances will generally result in lesser performance than that achieved by a shared instance. To reiterate, this statement is made without referring to any data supporting it. As such, the statement made in the documentation should perhaps be taken *cum grano salis*[1]. One improvement possibility for the documentation would therefore perhaps be to make this statement slightly more nuanced, by describing under which circumstances a shared instance performs better than private ones. Furthermore, the addition of data supporting this claim would also be very helpful, so much so even, that this work would perhaps be unnecessary.

In light of the performance results presented in this work, and the lack of development support for parallel producers, such as exists for consumers in the form of the Confluent Parallel Consumer API, a library which aims to provide such support has been developed as part of this work. The library is only a wrapper-library based on an implementation of a Kafka client library, and therefore provides only limited development support. For additional information regarding the library and its implementation, see Appendix B.

7.2 Future Work

One possibility for future work to build upon the work presented here would be to carry out a similar performance examination of parallel execution, but with a focus

[1] With a grain of salt

on what effect different network conditions have on this performance. Similarly, an examination of how this parallel performance depends on the number of brokers, topics, and partitions would likely also provide useful insight, as would the addition of Kafka consumers, executing at the same time as the producers.

Specifically, one important extension to the work presented here regards the number of **brokers** used when carrying out the performance evaluations. Presently, only a single broker, with the default configuration options, has been used. However, while using a single broker is perfectly allowed, and rather common for proof-of-concept uses of Kafka, the recommended minimum number of brokers for production systems seems to be 3, as this allows for a higher configured replication factor, resulting in higher cluster reliability [40] [39]. As such, utilizing a minimum of three brokers, ideally hosted on three separate machines, would perhaps produce data more representative for how Kafka is actually used in production systems. Additionally, as producers can write to different **partitions** in parallel, it may be the case that the introduction of additional partitions, hosted on the additional brokers, could affect the performance of the producers executing in parallel. Precisely what this effect would be is difficult to predict, but one guess could be that as messages would be spread out among the available brokers and written to them in parallel, any performance bottleneck caused by a single executing broker would likely be removed. As such, it may be the case that the performance improvements of private producers, compared with the performance of a shared instance, would only increase further when additional brokers are used. These expansions on the work would yield data more representative of the performance in actual production environments.

The work presented here has examined what can be seen as the two extremes on the scale of producer instances per thread: a single instance and one instance per thread. However, it may be the case that the best performance is achieved by some intermediary point on this scale, where several producer instances are used, by each such instance is used by multiple threads. Similar additions can also be done for the method chosen for enabling parallel execution. The use of OpenMP that has been made in this work has been rather simple, in order to keep the work focused and its scope reasonable. Therefore, there is some potential for examining and perhaps also improving upon the mechanism of parallelism used to execute producers concurrently. One possible improvement could be to examine a mechanism of parallelism entirely separate from OpenMP, such as determining whether it is possible to alter the implementation of the producers on a lower level, such that a single instance can make use of multiple CPU cores. How OpenMP is used could also be expanded upon, perhaps by examining the performance effects caused by using different scheduling types.

As the results that have been presented indicate that significant performance improvements may be realised by executing producer applications in parallel with thread-private producer instances, developing a library, larger than Brod, offering functionality similar to that offered by the Confluent parallel consumer, could offer a substantial amount of future work.

References

[1] Apache kafka - powered by. https://kafka.apache.org/powered-by. Accessed: 2023-04-17.

[2] Confluent parallel consumer. https://github.com/confluentinc/parallel-consumer. Accessed: 2023-04-15.

[3] Future shock: 11 real-life technologies that science fiction predicted. https://www.micron.com/insight/future-shock-11-real-life-technologies-that-science-fiction-predicted. Accessed: 2023-02-28.

[4] IBM documentation. https://www.ibm.com/docs/en/txseries/8.2?topic=overview-what-is-distributed-computing. Accessed: 2023-03-08.

[5] Individuals using the internet (% of population) | data. https://data.worldbank.org/indicator/IT.NET.USER.ZS. Accessed: 2023-02-28.

[6] Introduction to librdkafka - the apache kafka c/c++ client library. https://docs.confluent.io/platform/current/clients/librdkafka/html/md_INTRODUCTION.html#latency-measurement. Accessed: 2023-05-10.

[7] Kafka consumer configurations for confluent platform | confluent documentation. https://docs.confluent.io/platform/current/installation/configuration/consumer-configs.html. Accessed: 2023-02-02.

[8] Kafka producer configurations for confluent platform | confluent documentation. https://docs.confluent.io/platform/current/installation/configuration/producer-configs.html. Accessed: 2023-02-02.

[9] librdkafka - configuration properties. https://github.com/confluentinc/librdkafka/blob/master/CONFIGURATION.md. Accessed: 2023-04-21.

[10] Messaging that just works — RabbitMQ. https://www.rabbitmq.com/. Accessed: 2023-02-06.

[11] Refrigerators, smart fridges & freezers. https://www.samsung.com/us/refrigerators/. Accessed: 2023-03-01.

[12] Stackoverflow - how do i make kafka producer generate events in parallel? https://stackoverflow.com/questions/53302592/how-do-i-make-kafka-producer-generate-events-in-parallel. Accessed: 2023-04-15.

[13] State of IoT 2022: Number of connected IoT devices growing 18% to 14.4 billion globally. https://iot-analytics.com/number-connected-iot-devices/. Accessed: 2023-03-01.

[14] Table 1. system performance comparison for old and new architectures. https://blog.twitter.com/engineering/en_us/topics/infrastructure/2021/processing-billions-of-events-in-real-time-at-twitter-. Accessed: 2023-03-01.

[15] Vehicle telematics, geofencing & fleet tracking systems for the connected car. http://www.airbiquity.com/. Accessed: 2023-03-01.

[16] Berker Agir. Benchmarking kafka producer throughput with quarkus. https://developers.redhat.com/articles/2021/07/19/benchmarking-kafka-producer-throughput-quarkus. Accessed: 2023-04-19.

[17] Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities, reprinted from the afips conference proceedings, vol. 30 (atlantic city, n.j., apr. 18–20), afips press, reston, va., 1967, pp. 483–485, when dr. amdahl was at international business machines corporation, sunnyvale, california. *IEEE Solid-State Circuits Society Newsletter*, 12(3):19–20, 2007.

[18] Mark Bohr. A 30 year retrospective on dennard's mosfet scaling paper. *IEEE Solid-State Circuits Society Newsletter*, 12(1):11–13, 2007.

[19] Dennis M. Ritchie Brian W. Kernighan. *The C programming language*. Prentice Hall, 2nd edition.

[20] Jörg Keller Christoph Kessler. Models for parallel computing: Review and perspectives. 12 2007.

[21] Confluent. Apache kafka performance - benchmark testing and results for apache kafka's performance on the latest hardware in the cloud. https://developer.confluent.io/learn/kafka-performance/. Accessed: 2023-04-19.

[22] Confluent. Batch processing for efficiency. https://docs.confluent.io/kafka/design/efficient-design.html#batch-processing-for-efficiency. Accessed: 2023-04-02.

[23] George Coulouris, Jean Dollimore, Tim Kindberg, and Gordon Blair. *Distributed Systems: Concepts and Design*. Pearson, 5th edition edition.

[24] John L. Hennessy David A. Patterson. *Computer Organization and Design - ARM Edition*. Morgan Kaufmann, 1st edition.

[25] Flavien Raynaud Nikhil Benesch David Blewett, Federico Giraud. rust-rdkafka - apache kafka client library for rust. https://crates.io/crates/rdkafka. Accessed: 2023-05-04.

[26] R.H. Dennard, F.H. Gaensslen, Hwa-Nien Yu, V.L. Rideout, E. Bassous, and A.R. LeBlanc. Design of ion-implanted mosfet's with very small physical dimensions. *IEEE Journal of Solid-State Circuits*, 9(5):256–268, 1974.

[27] Shantanu Deshmukh. Message compression in apache kafka. https://developer.ibm.com/articles/benefits-compression-kafka-messaging/. Accessed: 2023-04-16.

[28] Philippe Dobbelaere and Kyumars Sheykh Esmaili. Kafka versus rabbitmq. https://arxiv.org/abs/1709.00333, 2017.

[29] Magnus Edenhill. librdkafka - the apache kafka c/c++ client library. https://github.com/confluentinc/librdkafka. Accessed: 2023-04-21.

[30] Pekka Enberg. On parallel programming. https://penberg.org/blog/parallel.html. Accessed: 2023-04-17.

[31] The Apache Software Foundation. Apache kafka. https://kafka.apache.org/. Accessed: 2023-04-15.

[32] The Apache Software Foundation. Apache kafka - main concepts and terminology. https://kafka.apache.org/documentation/#gettingStarted. Accessed: 2023-02-01.

[33] The Apache Software Foundation. Documentation - kafkaproducer¡k,v¿. https://kafka.apache.org/0110/javadoc/org/apache/kafka/clients/producer/KafkaProducer.html. Accessed: 2023-04-15.

[34] The Rust Foundation. Rust programming language. https://www.rust-lang.org/. Accessed: 2023-05-09.

[35] M. Fowler. Design - who needs an architect? *IEEE Software*, 20(5):11–13, 2003.

[36] Inc. Free Software Foundation. Using the gnu compiler collection - options that control optimization. https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html. Accessed: 2023-05-17.

[37] Donald Gross. *Fundamentals of Queueing Theory*. Wiley-Blackwell, 4th edition.

[38] Sören Henning and Wilhelm Hasselbring. Benchmarking scalability of stream processing frameworks deployed as event-driven microservices in the cloud, 2023.

[39] Lovisa Johansson. Apache kafka replication factor - what is the perfect number? https://www.cloudkarafka.com/blog/apache-kafka-replication-factor-perfect-number.html. Accessed: 2023-05-10.

[40] Conductor Kafkademy. Kafka topics - choosing the replication factor and partition count. https://www.conduktor.io/kafka/kafka-topics-choosing-the-replication-factor-and-partitions-count/. Accessed: 2023-05-10.

[41] Carl Leche. Tokio - an asynchronous rust runtime. tokio.rs. Accessed: 2023-05-09.

[42] Claudia Loebbecke and Arnold Picot. Reflections on societal and business model transformation arising from digitization and big data analytics: A research agenda. *The Journal of Strategic Information Systems*, 24(3):149–157, 2015.

[43] Patrick Mikalef, Ilias Pappas, John Krogstie, and Paul Pavlou. Big data and business analytics: A research agenda for realizing business value.

[44] Gordon Moore. *Cramming More Components onto Integrated Circuits (1965)*, pages 261–266. 02 2021.

[45] Pramod Sadalage Zhamak Dehghani Neal Ford, Mark Richards. *Software Architecture: The Hard Parts: Modern Trade-Off Analyses for Distributed Architectures*. O'Reilly Media, 1 edition, 2021.

[46] Prashant Pandey. How to create a multithreaded producer in java. `https://www.learningjournal.guru/article/kafka/how-to-create-a-multithreaded-kafka-producer-in-java/`. Accessed: 2023-04-15.

[47] Matthew Malensk Peter Pacheco. *An Introduction to Parallel Programming*. Morgan Kaufmann, 2nd edition.

[48] Rob Pike. Concurrency is not parallelism. `https://www.youtube.com/watch?v=oV9rvDllKEg&t=1s&ab_channel=gnbitcom`. Accessed: 2023-04-17.

[49] Quiency Koziol Prabhat. *High Performance Parallel I/O*. 1st edition.

[50] Theofanis P. Raptis and Andrea Passarella. On efficiently partitioning a topic in apache kafka. In *2022 International Conference on Computer, Information and Telecommunication Systems (CITS)*, pages 1–8. Accessed: 2023-02-03.

[51] Andrea C. Arpaci-Dusseau Remzi H. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books, 1st edition.

[52] Yuliana Zamora Robert Robey. *Parallel and High Performance Computing*. Manning, 1st edition.

[53] Amazon Web Services. Amazon ec2- secure and resizable compute capacity for virtually any workload. `https://aws.amazon.com/ec2/`. Accessed: 2023-05-11.

[54] Amazon Web Services. Cloud computing services- amazon web services. `https://aws.amazon.com/`. Accessed: 2023-05-11.

[55] Gwen Shapira, Todd Palino, Rajini Sivaram, and Krit Petty. *Kafka: The Definitive Guide*. O'Reilly Media, 2nd edition edition.

[56] Arvid Sundbom. Brod. `crates.io/crates/brod`. Accessed: 2023-05-04.

[57] Arvid Sundbom. Kafka-openmp. `https://github.com/suarvid/kafka-openmp`. Accessed: 2023-06-16.

[58] Arvid Sundbom. Producer application utilizing multiple threads. `https://lists.apache.org/thread/g3zl6c8x3ysrc8p5gvlr35d0bpn4zd96`. Accessed: 2023-05-11.

[59] Ruud van der Pas. Mastering openmp performance. https://www.openmp.org/wp-content/uploads/openmp-webinar-vanderPas-20210318.pdf. Accessed: 2023-04-17.

[60] M. van Steen and A.S. Tanenbaum. *Distributed Systems*. 4th edition edition.

[61] Andrej Walilko. The difference between cpu cores and threads. https://www.liquidweb.com/blog/difference-cpu-cores-thread/. Accessed: 2023-05-11.

[62] Wikipedia. Max brod. https://en.wikipedia.org/wiki/Max_Brod. Accessed: 2023-05-09.

A Kafka Configuration Values

Table 5 contains the concrete numerical values corresponding to the discrete values *Default*, *Medium*, and *Maximum*, in the Kafka profiles that were tested. In all cases, the value assigned to the Default setting is the default value assigned when creating a producer without explicitly configuring the value, when using the librdkafka library [9].

Table 5 The numerical values corresponding to the levels Default, Medium, and Maximum, for three Kafka producer settings which have a large impact on the throughput achieved by producers.

	Linger Time (ms)	Batch Size (bytes)	Queue Buffering (kbytes)
Default	5	1000000	1048576
Medium	1000	10000000	10485760
Maximum	900000	2147483647	2147483647

B Brod - Parallel Producer Support

Brod [56], named after Franz Kafka's friend *Max Brod* [62], is a wrapper-library written in Rust [34], based on the Rust Kafka client library *rdkafka* [25]. The purpose of the library is to provide support for developing multi-threaded Kafka producer applications in which each thread is assigned its own producer instance. This support is provided via an API which provides functionality which aims to enable developers to write functions intended to utilize a producer instance in a sequential manner, and then have this function execute in parallel.

This API consists of two main parts, one which executes a provided "worker function" in parallel via a "wrapper function", and one which provides a pool of producer instances, changing which instance is used to produce messages each time a message is sent. For either case, both synchronous and asynchronous implementations are provided, using synchronous and asynchronous producer instances, respectively. Common to the use of all these variants is the requirement of supplying a producer configuration, used to create the underlying instances, and a topic to which the messages are sent. For additional information regarding the use of the library, see the **documentation** and the provided **examples**.

B.1 Wrapper Function

Use of the wrapper function is based on function pointers. In both the synchronous and asynchronous case, two APIs are available for constructing a wrapper function based on such a function pointer. One is based on generic functions, and one on declarative macros.

Generic Function. The first of these makes use of generic functions and *type-erasure* in order to achieve the necessary level of flexibility required in order for the wrapper function to be generally applicable. This API expects that a pointer to a "worker function" is passed to the respective "wrapper function". In addition to this function pointer, the wrapper function also requires a producer-configuration object, from which it may create the underlying instances used to produce messages.

In order to adhere to the expected interface, the worker function is itself expected to take the following as its arguments when being called: a producer instance, a string slice containing the name of the topic to which messages are to be produced, as well as a vector of type `Vec<Arc<dyn Any + Sync + Send>>` i.e. a vector of *type-erased* arguments that can be passed between threads. This allows users of the library to pass (almost) any arguments to the worker function, which may be necessary as the worker function is also expected to be responsible for sending all messages

and performing any computation required to do so. These arguments also need to be passed to the wrapper function, which in turn passes them to the worker function.

A generic type parameter, `T`, is used in order to enable the worker function to return a value of any type. As the worker function is intended to be executed by multiple threads, multiple return values will be produced. As such, these return values are combined into a vector of type `Vec<T>` in order to be returned from the wrapper function. However, as it is possible that some error occurs either when creating the producer instances or when sending messages, this vector is itself wrapped in an instance of Rust's `Result` enum. This enum has two variants, one representing a successful computation, denoted `Ok`, and one representing a failed computation, denoted `Err`. Both variants each contain a value, representing the result of the successful computation or information regarding the error, respectively. As such, the type of the value returned by the wrapper function is `Result<Vec<T>, Box<dyn Any+ Send>>`; again, type erasure is used in order to allow a value of any type to be returned when an error occurs.

The API for synchronous producers also exposes a variant of the parallel execution wrapper-function which returns an instance of the Result enum which, if no error occurs, contains a vector of type Vec<Box<dyn Any + Sync + Send>>. This is another instance where the type-erasure mechanism is used, in this case to allow the wrapper-function to return multiple values of differing types.

The main drawback of using type-erasure is the fact that it reduces the amount of work done by the compiler, and instead requires that this work be done by the developer. As the name suggests, type-erasure removes useful type information normally available in values. Therefore, this work mainly consists of type-checking, specifically ensuring that the types of the values sent to or returned from these functions are valid according to the way these values are used by the program. Additionally, the developer also needs to write additional code for type-casting these values, in order to erase and re-introduce the types associated with these values. Here, the compiler can at least be of assistance by ensuring that any eventual casting failures are handled, in some way.

The part of the library which supports asynchronous producers introduces two major complications: the runtime necessary for supporting asynchronous execution, and, in the case of the asynchronous wrapper function, rather complicated code required for supporting function pointers to generic asynchronous functions. The former of these complications is relatively simple to fix; in this case, the external library tokio is used, which includes a runtime which may be used for asynchronous execution [41]. As such, asynchronous producers are supported via the simple addition of a few annotations. Supporting pointers to generic asynchronous functions is not as simple. Just as in the synchronous case, the asynchronous worker function is expected to take a producer instance, a string slice containing a topic name, and a vector of additional, type-erased arguments, as its parameters. In order to enable this support, a struct, AsyncFnWrapper, has been created, which represents a wrapper for a pointer to an asynchronous function. Because of Rust's advanced and strict type system and compiler, creating a new instance of this wrapper type is no easy task.

Currently, instances can be created by calling the new function associated to the type. However, this function requires, as its argument, a closure (anonymous function) which itself should take the aforementioned producer, topic, and additional arguments, as parameters. This closure should wrap the "regular" pointer to the worker function behind a pointer of type Pin<Box<_>>, i.e. a pointer to a value which may not be relocated in memory, and return this wrapped pointer. As such, utilizing the library with asynchronous producers requires some additional development effort. However, the library provides some utility functions in order to reduce this effort.

Declarative Macro. The second approach to constructing a wrapper function based on a pointer to a worker function makes use of Rust's *declarative macros*. As a brief description of what macros are in a programming context, they are a language construct that map a given input to a given output, which is commonly referred to as *expanding* the macro. A typical application of this mapping mechanism is for it to be used by a program upon the source code of the program itself, altering the program as it is being constructed. This phenomenon of a program which is programmed to alter itself is called *metaprogramming*.

In languages like C, macro expansion is carried out in a fairly simple manner, early on in the compilation process. Essentially, this expansion boils down to replacing one piece of text, called a *token*, with another, namely the definition corresponding to the invoked macro. However, in Rust, macro expansion is deferred to a later stage in the compilation process, specifically *after* the *Abstract Syntax Tree (AST)* has been constructed. As a somewhat simplified description, this allows macros in Rust to be more advanced and *expressive*, at the cost of increased complexity.

Because of this, Rust can provide a mechanism called *declarative macros*, also referred to as *macros by example*. Such macros are constructed by utilizing a mechanism of *pattern-matching*. When a macro is invoked, it will (optionally) be provided with a set of arguments. The compiler then attempts to match these arguments with one of the patterns constituting the body of the macro, expanding the matched arguments to the code in the corresponding part of the macro body. The structure of the definition of a declarative macro is shown in Listing B.1. Additionally, the form of each matching rule shown in the listing is `($pattern_to_match) => {$expansion}`

Listing B.1: The structure of the definition of a declarative macro

```
macro_rules! $macro_name {
    $rule_0 ;
    $rule_1 ;
    ...
    $rule_n ;
}
```

Declarative macros can be used to dynamically construct code during compilation, depending on which arguments are passed in the macro invocation. As such, declarative macros can be employed as a powerful tool for constructing functions which, in turn, utilize function pointers to functions with unspecified signatures. Simplifying somewhat, for each invocation of the macro, the compiler effectively generates code

for a new function, containing code which calls the function passed as an argument with the corresponding arguments provided in the macro invocation.

This is the approach used in Brod; for each worker function passed to the `produce_parallel!` macro, code is generated for calling that function with the supplied arguments. The main advantage of this approach is that this code generation, and therefore the associated type-checking, is carried out during compilation. As such, type-erasure is not necessary for achieving flexibility in how the API may be used in client code. In turn, this removes both the (small) overhead incurred by carrying out type-conversions at runtime, as well as the additional code users are required to write for casting the worker function's type-erased arguments back to their intended types. Additionally, fewer constraints are placed on the worker functions and the use of the macro-based wrapper function. The only requirements are that the number of threads to be used for execution and the producer configuration to be used to create producer instances are passed to the wrapper macro, and that the arguments to be passed to the worker function match its function signature.

B.2 Producer Pool

Another way in which the library can be used in order to support executing producer applications in parallel is by the use of the producer pool structure exposed in the API, either synchronously or asynchronously, in the form of `SyncProducerPool` and `AsyncProducerPool`, respectively. While not offering functionality for parallel execution on its own, the intent behind this data structure is to be shared among multiple threads of execution in code which is made parallel by the development efforts of the user of the library. Each time the pool is used to send data, the producer instance used in the next request to send data is changed, in a round-robin fashion. As such, the producer pools may offer an alternative for increasing application performance while giving developers more control and requiring only a small amount of additional development effort, assuming the mechanism for parallel execution has already been implemented.

Just as for the wrapper functions, instantiating a producer pool requires passing a producer-configuration object, from which the producer instances will be created. Furthermore, just as when producing messages with a "regular" producer instance, a topic, and a message payload are required to produce messages, with optional keys and headers that may be added as well.

B.3 Performance

This Section gives a brief overview of the performance data measured in various uses of the library. All of the scenarios used to collect the performance data presented in this Section are included in the examples in the library repository.

An overview of the performance achieved by passing a function pointer to a worker

function, which makes use of type-erasure, as an argument to the synchronous wrapper function ‘produce_in_parallel‘ is shown in Table 6. The task used to measure the shown execution times is simply to send 100 000 messages, where each message has a key and payload, both being short strings. In the case of this test, both the producer application and the broker were hosted on the same machine. As can be seen in the Table, when only a single thread, or a pair of threads, are used to send the messages, using a single thread and producer instance results in better performance than either sharing a producer among threads or using the wrapper function supplied by the library. However, as the number of threads increases, the execution time of the wrapper function decreases significantly. On the other hand, increasing the number of threads sharing the producer instance always results in a longer execution time than sequential execution. Furthermore, as the number of threads reaches larger values, the execution time increases drastically, due to messages entering the internal producer queue at a higher rate than the producer is capable of dispatching messages.

Table 6 Execution times, rounded to the nearest millisecond, when sending 100 000 messages with synchronous producer instances

Number of Threads	Sequential	Parallel, Shared Producer	Wrapper Function
1	50	58	130
2	46	111	71
4	43	154	22
8	44	370	16
12	45	813	10
24	42	4094	7
48	42	4807	5

Furthermore, comparing the performance of the wrapper function based on type-erasure and the one based on a macro may also be of interest, as the latter attempts to achieve superior performance by removing the need for type-conversions at runtime. The execution times measured with these two approaches, for a number of thread counts, are shown in Table 7. As can be seen by the execution times presented in the Table, initially, the macro-based implementation achieves shorter execution times than the implementation based on type-erasure. However, as the thread count increases, the performance of the macro-based implementation begins to degrade, while the the one utilizing type-erasure performs better and better.

A possible explanation for this can be found by observing the values in the two leftmost columns. These columns contain the percentage of the sent messages which were processed by a console-based Kafka consumer, which simply prints the data it reads to the console, and also counts the number of messages it has processed. Both implementations were executed utilizing a producer instance configured for high throughput, without either acknowledgement messages nor EOS. As a result of this, both implementations produce message counts which significantly differ from the number of messages they are actually supposed to produce. However, this difference is significantly larger when the function-based implementation is executed compared to the macro-based one, with around 90 percent of the intended messages failing to

be processed by the consumer, and therefore failing to reach the topic hosted by the broker. As such, although the measured execution time is significantly lower for the function-based implementation, this reduction comes at a great cost, and is arguably not worth the loss of data it incurs.

Table 7 Execution times, rounded to the nearest millisecond, and percentage of messages processed by a consumer, for one million messages sent with the two implementations of the synchronous wrapper function

Threads	Time, Function	Time, Macro	Processed, Function,	Processed, Macro
1	1648	708	561%	483%
2	798	449	277%	93%
4	409	316	155%	86%
8	207	379	64%	84%
12	105	462	34%	77%
24	177	410	9%	62%
48	27	453	12%	42%

As previously mentioned, the library also exposes an interface for using a collection of producer instances, referred to as a "pool", where the particular instance used to send data changes with each function call for producing a message. Table 8 contains data collected by running a simple performance test, in which one million messages were sent to brokers hosted on the same machine as the producer application.

As can be seen in the Table, use of the producer pool does, under some circumstances, result in better performance than using a single producer instance, either with a single thread or shared between multiple ones. However, this performance improvement is rather limited compared to the performance improvements achieved by using the wrapper function; while use of the wrapper function leads to significantly reduced running times in all cases where four or more threads are used, the performance of the producer pool only really outperforms the other configurations when using 2, 4 or 8 threads, and only does so to a fairly limited extent. One possible explanation for this is the overhead added by the mechanism used to ensure that race conditions do not occur in the internals of the pool data structure. As the producer pool is arguably easier to use and integrate into existing code however, it may be useful for applications that are not performance-critical, but could benefit from slightly improved performance.

Table 8 Execution times, in milliseconds, when sending 1 000 000 messages with a synchronous producer pool

Number of Threads	Sequential	Parallel, Shared Producer	Producer Pool
1	1013	951	1073
2	898	831	714
4	1098	787	570
8	946	842	806
12	877	996	874
24	944	1451	1272
48	988	1882	1413